GUÍA TÉCNICA DE LA PLATAFORMA

# STYLUS

## Retail de moda con compras inteligentes asistidas por IA en tiempo real

*Arquitectura, pipelines, modelo de datos, operación, seguridad y costos en Google Cloud*

| Campo | Detalle |
|---|---|
| **Curso** | UTEC · Taller de Arquitectura Big Data · Proyecto final |
| **Documento** | Guía técnica de la plataforma (versión consultoría) |
| **Versión / fecha** | v1.0 · 24 de octubre de 2026 |
| **Fuente de datos** | Dataset público **H&M Personalized Fashion Recommendations** (Kaggle) |
| **Proyecto GCP** | `tranquil-leaf-512618-b9` · región `us-central1` · ambiente `dev` |
| **Repositorio** | `github.com/estuardoo/STYLUS` (privado) |
| **Clasificación** | Uso académico · no contiene credenciales ni datos de clientes |

| Integrante | Rol en el proyecto |
|---|---|
| **Estuardo Oliver** | **ML / Analytics** · Chatbot con Vertex AI Gemini, analítica, recomendaciones; integra documentos y demo. |
| **Winston Quispe** | **Cloud Computing** · Todo lo nube: Terraform, IAM, GCS, Pub/Sub, Dataflow (ingesta streaming), CI/CD, costos, DRP. |
| **Nicole Arenas** | **Datos + API + Modelado** · Bronze/Silver/Gold, calidad, modelo estrella, FastAPI y base transaccional. |
| **Gianmarco Mejia** | **Dashboard / BI** · KPIs, Looker Studio, calculadora y tablero de costos, diagramas y slides. |

# Control del documento

| Versión | Fecha | Autor | Cambios |
|---|---|---|---|
| v0.1 | 23-oct-2026 | Estuardo Oliver | Guía resumida (7 páginas) generada desde Markdown |
| v1.0 | 24-oct-2026 | Estuardo Oliver | Reescritura en formato consultoría: diagramas por pipeline, RACI, riesgos, costos, rutas reales y anexos de esquemas |

### Cómo leer esta guía

Los capítulos **1 a 10** explican cada tema (qué se construyó, por qué y quién lo hizo). Los **anexos A a H** contienen el detalle completo: rutas, tablas, columnas, esquemas estrella, permisos y parámetros.

Todo nombre de recurso, ruta y cifra proviene del repositorio real (`config/control_plataforma.json`, `docs/gestion/Matriz_Auditoria_Proyecto.xlsx`, calculadora de costos). La guía se regenera con `python docs/_generadores/build_guia_data.py` y `node docs/_generadores/build_guia.js`.

Convención de estado: ✔ **Hecho** · ● **Listo para desplegar** (código probado en local, falta ejecutarlo en GCP) · ✖ **Pendiente**.

# Índice

**1. Resumen ejecutivo 4**
1.1 Mensajes clave 4
1.2 Recomendación 4
**2. Contexto, alcance y fuente de datos 5**
2.1 Objetivo del proyecto 5
2.2 Fuente de datos: dataset público H&M 5
2.3 Cronograma 6
**3. Equipo, roles y gobierno 7**
3.1 Perfiles y responsabilidades 7
3.2 Matriz RACI por dominio 7
3.3 Qué hace cada persona en la Entrega 1 8
3.4 Modelo de gobierno 9
**4. Arquitectura de la solución 10**
4.1 Decisiones por capa 11
4.2 Principios de diseño 11
4.3 Nombres reales de los recursos 11
**5. Pipelines de datos 12**
5.1 Pipeline A — Batch (Bronze → Silver → Gold) 12
5.2 Pipeline B — Streaming (replay 2026) 14
5.3 Pipeline C — Servicio (API, asistente y tienda) 15
5.4 Pipeline D — Control de configuración y CI/CD 16
**6. Modelo de datos 17**
6.1 Modelo estrella (Gold) 17
6.2 Vistas para tableros 17
6.3 Recomendador 18
**7. Configuración y operación 19**
7.1 Fuente única: config/control_plataforma.json 19
7.2 Panel de control web 19
7.3 Orquestador ops.py 19
7.4 Procedimiento de despliegue 20
**8. Seguridad, identidad y gobierno de datos 21**
8.1 Cuentas de servicio (mínimo privilegio) 21
8.2 Secretos 21
8.3 Controles 21
**9. Costos y FinOps 22**
9.1 Palancas de ahorro 23
**10. Calidad, estado y hoja de ruta 24**
10.1 Pruebas automatizadas 24
10.2 Estado por entrega 24
10.3 Riesgos y mitigación 26
10.4 Próximos pasos 26
**Anexo A. Inventario de rutas del repositorio 27**
**Anexo B. Catálogo de tablas y vistas 29**
**Anexo C. Diccionario de columnas 30**
**Anexo D. Modelo estrella (Gold) 38**
**Anexo E. Matriz de permisos IAM 39**
Usuarios iniciales 40
**Anexo F. Parámetros de configuración 41**
**Anexo G. APIs de Google Cloud habilitadas 43**
**Anexo H. Glosario 44**

# 1. Resumen ejecutivo

**STYLUS** es una plataforma de datos sobre **Google Cloud** que convierte el histórico público de compras de H&M en tres capacidades de negocio: **analítica** (lakehouse medallion y tableros), **personalización** (recomendador con +200% de precisión sobre la línea base) y **comercio asistido** (tienda web con asistente Vertex AI Gemini y billetera transaccional). Toda la infraestructura se declara como código y se controla desde un único archivo de configuración.

| 31.8 M | +200% | USD 0.00098 | 81% |
|---|---|---|---|
| transacciones reales procesadas | MAP@12 vs. popularidad | costo por pedido en producción | Entrega 1 completada |

## 1.1 Mensajes clave

- **La arquitectura está completa y probada en local**: 51 tareas planificadas, 20 hechas y 18 listas para desplegar (código y pruebas terminados, falta ejecutarlas contra GCP).
- **Una sola fuente de verdad**: `config/control_plataforma.json` define 64 parámetros, 30 tablas, 240 columnas, 30 permisos IAM y 12 usuarios; `ops.py` valida y genera todo lo demás, y un **panel web** permite editarlo sin tocar código.
- **Costo controlado**: USD 11.57/mes en demo y USD 370.83/mes en producción; presupuesto con alertas al 50/90/100% y costo real por pedido desde el billing export.
- **Seguridad por diseño**: sin datos ni credenciales en Git (lo bloquea la CI), secretos en Secret Manager generados por Terraform, despliegue desde GitHub con **Workload Identity Federation** (sin llaves).

## 1.2 Recomendación

> **Siguiente paso crítico**
>
> Ejecutar `python ops.py todo` (o los botones del panel) contra el proyecto `tranquil-leaf-512618-b9` desde Cloud Shell o una PC con gcloud. Ese único paso convierte **18 tareas listas** en tareas hechas y desbloquea la Entrega 2 (07-nov) y los tableros de la Entrega 3.
>
> Responsable sugerido: **Winston Quispe** (infraestructura) con apoyo de **Nicole Arenas** (cargas Bronze/Silver/Gold).

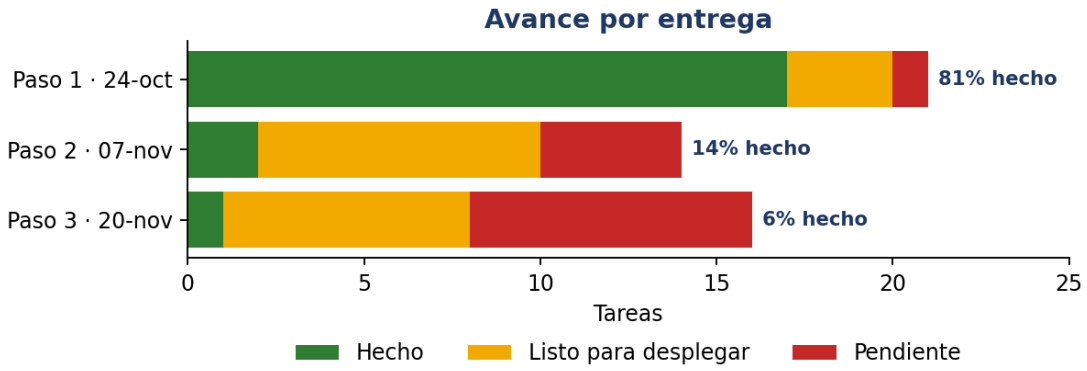



**Figura 1.** *Avance por entrega (fuente: matriz de auditoría).*

# 2. Contexto, alcance y fuente de datos

## 2.1 Objetivo del proyecto

Diseñar y desplegar una arquitectura Big Data de retail que soporte **ingesta batch y streaming**, **almacenamiento por capas**, **modelado analítico**, **exposición vía API** con un asistente de IA, y **control de costos automatizado**, cumpliendo las tres entregas del curso.

| Dentro del alcance | Fuera del alcance |
|---|---|
| Lakehouse Bronze/Silver/Gold en GCS + BigQuery | Pagos reales (el saldo de la tienda es simulado) |
| Replay de transacciones como eventos 2026 (Pub/Sub + Dataflow) | Datos de clientes reales distintos del dataset público |
| Recomendador baseline evaluado con MAP@12 | Modelos de deep learning entrenados a medida |
| API FastAPI + web + asistente Gemini + wallet ACID | Aplicación móvil |
| Infraestructura como código, CI/CD sin llaves, costos | Multirregión activa-activa (se documenta DRP) |

## 2.2 Fuente de datos: dataset público H&M

Todo el proyecto usa **exclusivamente** el dataset público de Kaggle **H&M Personalized Fashion Recommendations** (H&M Group, 2022). Los eventos de streaming de 2026 no son datos inventados: son **re-emisiones** de esas mismas transacciones con fecha 2026, marcadas con `is_synthetic = true` y conservando la fecha original en `source_t_dat`.

| Archivo original | Filas | Contenido | Ruta local → destino en GCS |
|---|---|---|---|
| `articles.csv` | 105,542 | Catálogo: tipo, color, sección, descripción | `data/raw/articles.csv` → `gs://tranquil-leaf-512618-b9-retail-raw/hm/` |
| `customers.csv` | 1,371,980 | Clientes: edad, membresía, frecuencia de noticias | `data/raw/customers.csv` → `gs://tranquil-leaf-512618-b9-retail-raw/hm/` |
| `transactions_train.csv` | 31,788,324 | Compras 2018-2020: cliente, artículo, fecha, precio, canal | `data/raw/transactions_train.csv` → `gs://tranquil-leaf-512618-b9-retail-raw/hm/` |
| `images/` | 105,100 | Fotos de producto (opcionales, `--con-imagenes`) | `data/raw/images/` → `gs://tranquil-leaf-512618-b9-retail-raw/hm/images/` (Nearline a los 30 días) |

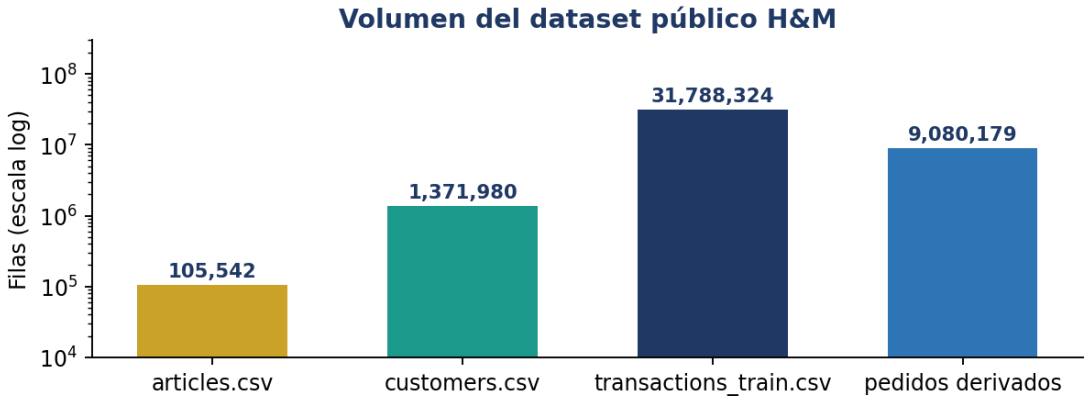



**Figura 2.** *Volumen del dataset (escala logarítmica).*

> **Hallazgos de la auditoría de datos (P1-01)**
>
> H&M **no trae identificador de pedido**: se define pedido = cliente + día + fuente (9,080,179 pedidos derivados).
>
> H&M no documenta los canales 1 y 2: se asumen **Tienda física** y **Online** (dimensión `dim_channel`, supuesto explícito).

> `article_id` tiene ceros a la izquierda: Bronze usa **esquema explícito STRING** para no perderlos.
> Los datos se usan según las reglas de la competencia; los CSV **nunca** se suben a GitHub (`data/` en `.gitignore` + regla de CI).

## 2.3 Cronograma




**Figura 3.** *Hitos del curso. La línea roja marca la fecha de esta versión.*

| Paso | Entrega | Fecha límite | Tareas | Hechas | Avance |
|---|---|---|---|---|---|
| Paso 1 | Examen + P1: diseño, costos, ingesta/storage | 24-oct-2026 | 21 | 17 | **81%** |
| Paso 2 | P2: pipelines batch + streaming, modelado | 07-nov-2026 | 14 | 2 | **14%** |
| Paso 3 | P3: API, asistente, tableros, demo, sustentación | 20-nov-2026 | 16 | 1 | **6%** |

# 3. Equipo, roles y gobierno

Cada integrante es **responsable (R)** de un dominio técnico y **apoya (A)** en los dominios vecinos. La asignación vive en la matriz de auditoría (`docs/gestion/Matriz_Auditoria_Proyecto.xlsx`), que se genera desde `docs/_generadores/build_matriz.py`.

## 3.1 Perfiles y responsabilidades

| Integrante | Perfil | Responsabilidad principal | Tareas (R) | Entregables clave |
|---|---|---|---|---|
| **Estuardo Oliver** | ML / Analytics | Asistente Vertex AI Gemini, recomendador y su evaluación, documentos de entrega, matriz de auditoría, guion de demo, publicación del repositorio y costos reales (D06, D19) | 11 | `api/assistant.py`, `pipelines/reco/baseline_eval.py`, `docs/entrega1/` |
| **Winston Quispe** | Cloud Computing | Terraform, IAM, Pub/Sub, Dataflow (streaming), CI/CD sin llaves, presupuesto y DRP, orquestador `ops.py` | 18 | `infra/terraform/`, `ops.py`, `pipelines/streaming/`, `.github/workflows/` |
| **Nicole Arenas** | Datos + API + Modelado | Auditoría de datos, Bronze/Silver/Gold, calidad, modelo estrella, FastAPI y Cloud SQL | 13 | `sql/`, `pipelines/batch/silver_hm.py`, `api/main.py`, `api/schema.sql` |
| **Gianmarco Mejia** | Dashboard / BI | Diagrama end-to-end, calculadora de costos, catálogo de KPIs, vistas BI, Looker Studio y slides | 9 | `docs/architecture/`, `cost/`, `sql/gold/02_kpi_views.sql` |

## 3.2 Matriz RACI por dominio

R = responsable de al menos una tarea del dominio; A = apoyo; R/A = ambas. Calculada automáticamente desde las 51 tareas.

| Dominio | EO · Estuardo | WQ · Winston | NA · Nicole | GM · Gianmarco |
|---|---|---|---|---|
| **Datos** | A | A | R | |
| **Gestión** | R | R/A | | R |
| **Seguridad** | | R | | |
| **Diseño** | A | R/A | R/A | R |
| **Costos** | | R/A | | R/A |
| **Resiliencia** | | R | A | |
| **BI** | A | A | A | R |
| **Infra** | | R | A | |
| **Modelado** | A | | R | |
| **Streaming** | | R | A | |
| **ML** | R | | A | |
| **API** | | A | R | A |

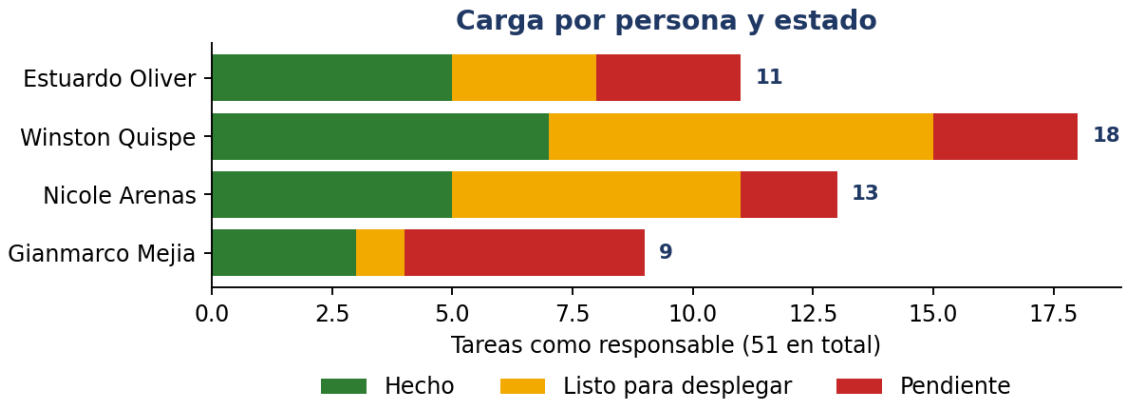



**Figura 4.** *Carga de trabajo por persona y estado.*

## 3.3 Qué hace cada persona en la Entrega 1

### Estuardo Oliver — 3/4 tareas hechas

| ID | Tarea | Qué hace | Entregable | Estado |
|---|---|---|---|---|
| P1-02 | **Ordenar carpeta y repositorio** | Estructura por capas; Amazon/zip archivados; secretos fuera | `README.md` | ✔ Hecho |
| P1-15 | **Documento Word Entrega 1** | Examen + P1 profesional | `docs/entrega1/*.docx` (+PDF) | ✔ Hecho |
| P1-16 | **Matriz de auditoría y roles** | Tareas, pasos, despliegue y variables | `docs/gestion/Matriz_Auditoria_Proyecto.xlsx` | ✔ Hecho |
| P1-19 | **Revisión y envío Entrega 1** | Revisión cruzada del Word y envío antes de 23:59 | `Word enviado` | ✖ Pendiente |

### Winston Quispe — 7/10 tareas hechas

| ID | Tarea | Qué hace | Entregable | Estado |
|---|---|---|---|---|
| P1-03 | **Proteger datos y secretos** | .gitignore + regla CI que bloquea CSV/ZIP/tfstate/credenciales | `.gitignore`, `.github/workflows/ci.yml` | ✔ Hecho |
| P1-04 | **Justificar stack GCP** | Decisión por capa con alternativa descartada | `Word §5` | ✔ Hecho |
| P1-08 | **Escalabilidad Black Friday** | Autoescalado, desacople, degradación controlada | `Word §8` | ✔ Hecho |
| P1-10 | **Automatizar control de costos** | Precios vía API, budget con alertas, costo real por pedido | `cost/`, `infra/terraform/main.tf` | ● Listo p/desplegar |
| P1-11 | **DRP / HA** | RPO/RTO por componente y runbook | `Word §10`, `docs/runbooks/drp.md` | ✔ Hecho |
| P1-14 | **Terraform base** | Buckets, BigQuery, Pub/Sub+DLQ, IAM, Artifact Registry, budget (valores desde el JSON de control) | `infra/terraform/` | ● Listo p/desplegar |
| P1-17 | **Crear proyecto GCP** | Proyecto, facturación/créditos, accesos IAM del equipo | `Proyecto GCP` | ✔ Hecho |
| P1-18 | **Publicar repositorio GitHub** | Crear repo privado, push y CI en verde | `URL del repo` | ✔ Hecho |
| P1-20 | **JSON de control de la plataforma** | Parámetros, APIs, cuentas, permisos, usuarios, datasets, tablas, columnas y secretos en un solo JSON validado, editable con el panel web | `config/control_plataforma.json` | ✔ Hecho |
| P1-21 | **Orquestador de despliegue** | Un comando por paso, multiplataforma, con log matricial de tiempos | `ops.py` | ● Listo p/desplegar |

### Nicole Arenas — 4/4 tareas hechas

| ID | Tarea | Qué hace | Entregable | Estado |
|---|---|---|---|---|
| P1-01 | **Auditoría de datos H&M** | Perfilar filas, nulos, fechas, cardinalidad y cobertura | `Word §4` | ✔ Hecho |
| P1-06 | **Integración multifuente** | Batch H&M + replay 2026; doble reloj source_t_dat/event_time | `Word §6` | ✔ Hecho |
| P1-07 | **Ingesta y almacenamiento** | GCS medallion + BigQuery + Cloud SQL; formatos y particiones | `Word §7` | ✔ Hecho |
| P1-13 | **Ingesta Bronze con manifiesto** | SHA-256 y filas por archivo; rsync idempotente | `ingestion/batch/upload_raw.py` | ✔ Hecho |

### Gianmarco Mejia — 3/3 tareas hechas

| ID | Tarea | Qué hace | Entregable | Estado |
|---|---|---|---|---|
| P1-05 | **Diagrama end-to-end** | Fuentes, ingesta, almacenamiento, cómputo, exposición | `docs/architecture/arquitectura.png` | ✔ Hecho |
| P1-09 | **Calculadora y costo por pedido** | YAML -> Excel con fórmulas; 4 escenarios; año 1 | `docs/entrega1/Calculadora_Costos_GCP.xlsx` | ✔ Hecho |
| P1-12 | **Catálogo de KPIs** | Metas de datos, streaming, modelo, API, negocio, costo | `Word §11` | ✔ Hecho |

## 3.4 Modelo de gobierno

- **Configuración**: un solo dueño por cambio del JSON de control; el diff del JSON generado se revisa en el pull request.
- **Infraestructura**: `plan` antes de `apply`; la CI nunca ejecuta `apply` ni `destroy`.
- **Datos**: ninguna fila se pierde en silencio — lo rechazado va a `quarantine/` y se concilia (entrada = salida + rechazadas).
- **Seguimiento**: la matriz de auditoría es la fuente del avance; esta guía y los gráficos se regeneran desde ella.

# 4. Arquitectura de la solución

La plataforma sigue una arquitectura **lakehouse medallion** con dos rutas de ingesta (batch y streaming) que convergen en un modelo estrella, y una capa de exposición que sirve tanto a analistas (Looker Studio) como a clientes (tienda web con asistente).

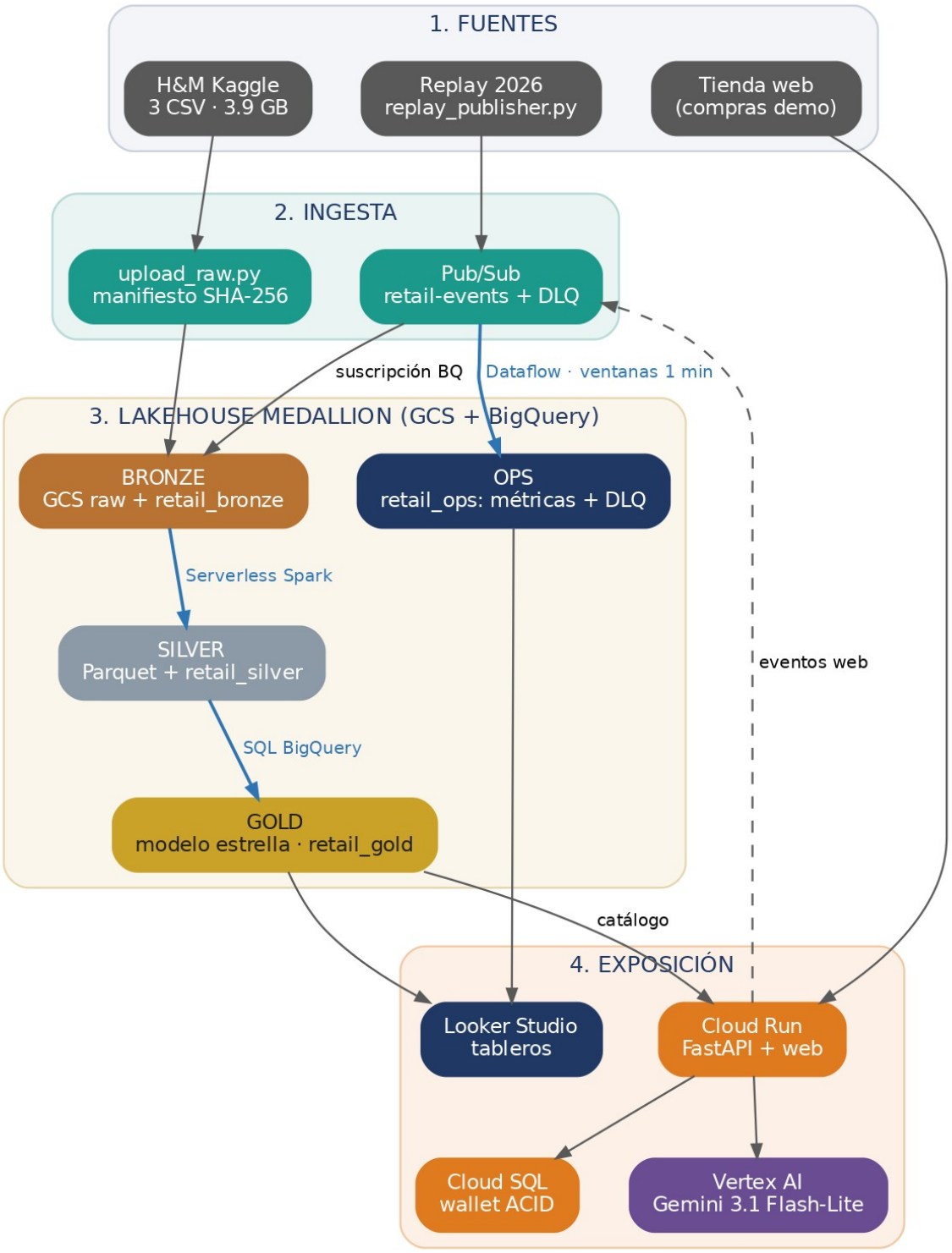



**Figura 5.** *Arquitectura end-to-end por capas: bronze, silver y gold; verde = ingesta; flechas azules = cómputo (Spark, SQL, Dataflow); naranja = exposición.*

## 4.1 Decisiones por capa

| Capa | Servicio elegido | Por qué | Alternativa descartada |
|---|---|---|---|
| **Almacenamiento** | **Cloud Storage** (raw, lake) | Barato, versionado, ciclo de vida a Nearline para imágenes | Disco en VM (sin durabilidad gestionada) |
| **Data warehouse** | **BigQuery** | Serverless, tablas externas sobre GCS, particiones y clustering, SQL estándar | Cluster Hadoop/Hive permanente |
| **Batch** | **Serverless for Apache Spark** | Spark sin administrar clúster; se paga por DCU-hora | Dataproc con clúster 24/7 |
| **Mensajería** | **Pub/Sub** + suscripción BigQuery + DLQ | Desacople, reintentos, ruta económica directa a BigQuery | Kafka autogestionado |
| **Streaming** | **Dataflow** (Apache Beam) | Ventanas, datos tardíos, deduplicación y métricas de lag gestionadas | Spark Streaming en clúster |
| **API / web** | **Cloud Run** | Escala a cero, contenedor estándar, HTTPS incluido | GKE (sobredimensionado para el alcance) |
| **Transaccional** | **Cloud SQL Postgres 15** | ACID para checkout y billetera append-only, PITR | Firestore (sin transacciones SQL multi-tabla) |
| **IA generativa** | **Vertex AI Gemini 3.1 Flash-Lite** | Misma IAM y facturación de GCP; costo bajo por conversación | API externa (otra llave y otra factura) |
| **BI** | **Looker Studio** | Gratis, conector nativo a BigQuery | Power BI / Tableau (licencias) |
| **IaC / CI** | **Terraform** + **GitHub Actions** + WIF | Reproducible, revisable, sin llaves en GitHub | Configuración manual en consola |

## 4.2 Principios de diseño

- **Idempotencia**: subir datos, re-ejecutar capas o re-publicar eventos no duplica resultados (`rsync`, `LOAD DATA OVERWRITE`, `event_id` determinista, `idempotency key` en checkout).
- **Trazabilidad del origen**: cada fila conserva `source`/`_source_system`, `is_synthetic` y la fecha original.
- **Configuración declarativa**: nombres, permisos y esquemas salen del JSON de control, nunca están escritos a mano en varios lugares.
- **Costo por defecto mínimo**: Cloud Run con 0 instancias mínimas, Dataflow solo durante la demo, Cloud SQL `db-f1-micro`.

## 4.3 Nombres reales de los recursos

| Recurso | Nombre en GCP |
|---|---|
| **Proyecto / región / ambiente** | `tranquil-leaf-512618-b9` · `us-central1` · `dev` |
| **Bucket Bronze (raw)** | `gs://tranquil-leaf-512618-b9-retail-raw` |
| **Bucket Silver/Gold (lake)** | `gs://tranquil-leaf-512618-b9-retail-lake` |
| **Estado de Terraform** | `gs://tranquil-leaf-512618-b9-tfstate/retail/dev` |
| **Datasets BigQuery** | `retail_bronze` · `retail_gold` · `retail_ops` · `retail_silver` |
| **Tópico Pub/Sub** | `retail-events` (+ DLQ) |
| **Imagen de la API** | `us-central1-docker.pkg.dev/tranquil-leaf-512618-b9/retail/retail-api` |
| **Servicio Cloud Run** | `retail-api` |
| **Modelo de IA** | `gemini-3.1-flash-lite` en `us-central1` |
| **Cuenta de servicio · api** | `sa-retail-api@tranquil-leaf-512618-b9.iam.gserviceaccount.com` |
| **Cuenta de servicio · deployer** | `sa-retail-deployer@tranquil-leaf-512618-b9.iam.gserviceaccount.com` |
| **Cuenta de servicio · ingest** | `sa-retail-ingest@tranquil-leaf-512618-b9.iam.gserviceaccount.com` |
| **Cuenta de servicio · pipeline** | `sa-retail-pipeline@tranquil-leaf-512618-b9.iam.gserviceaccount.com` |

# 5. Pipelines de datos

La plataforma tiene **cuatro pipelines**. Cada uno se ejecuta con un comando de `ops.py` (o un botón del panel) y deja evidencia verificable.

| Pipeline | Disparador | Entrada | Salida | Dueño |
|---|---|---|---|---|
| **A. Batch** (lakehouse) | `ops.py subir-datos · bronze · silver · gold` | CSV H&M | `retail_gold.*` | Nicole Arenas |
| **B. Streaming** | `ops.py replay · dataflow-iniciar` | Transacciones re-emitidas | `retail_silver.v_events`, `retail_ops.*` | Winston Quispe |
| **C. Servicio** (API) | `ops.py api · catalogo` | `retail_gold.v_catalog_serving` | Cloud Run + Cloud SQL | Nicole Arenas / Estuardo Oliver |
| **D. Control y CI/CD** | Edición del JSON · push a GitHub | `config/control_plataforma.json` | Terraform + despliegue | Winston Quispe |

## 5.1 Pipeline A — Batch (Bronze → Silver → Gold)

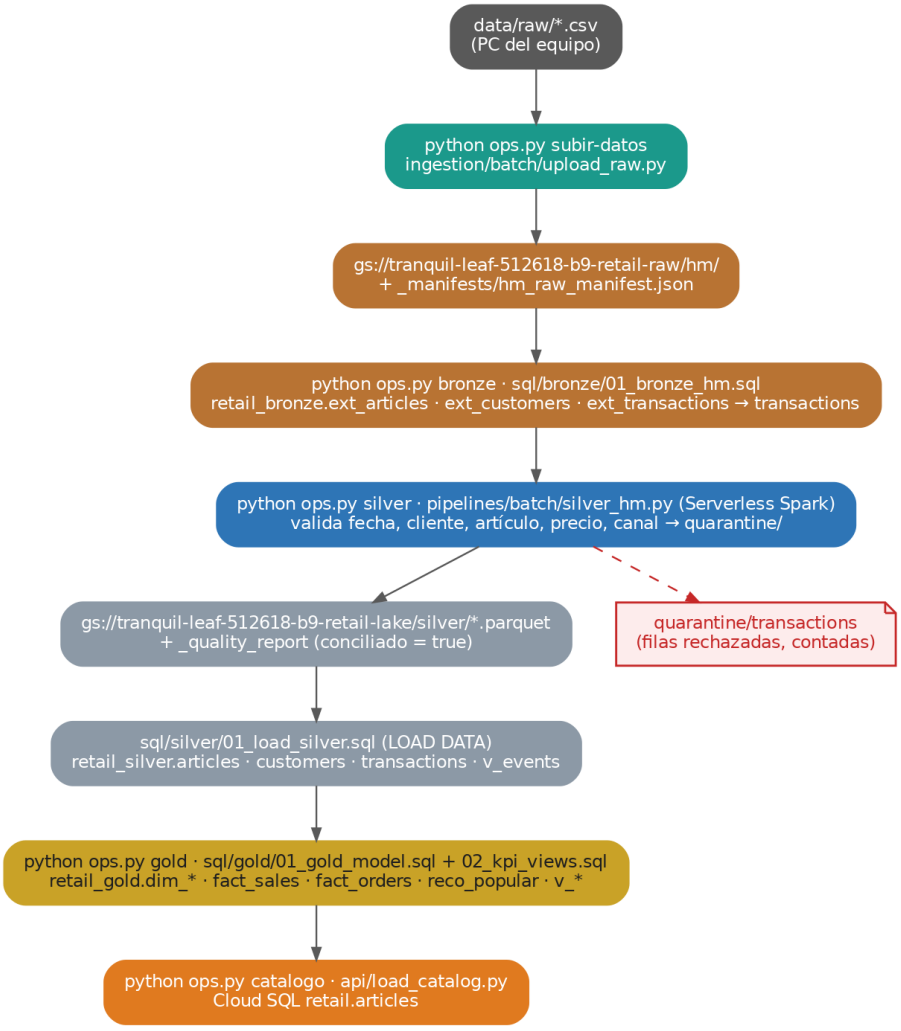



**Figura 6.** *Pipeline batch con comandos, rutas y tablas reales.*

| # | Etapa | Comando | Código | Qué hace | Resultado verificable |
|---|---|---|---|---|---|
| A1 | Carga a Bronze | `ops.py subir-datos` | `ingestion/batch/upload_raw.py` | SHA-256 y filas por archivo; `gcloud storage rsync` idempotente | `_manifests/hm_raw_manifest.json` |
| A2 | Bronze | `ops.py bronze` | `sql/bronze/01_bronze_hm.sql` | Tablas externas con esquema STRING explícito; copia nativa particionada por día | 31,788,324 filas en `retail_bronze.transactions` |
| A3 | Silver (Spark) | `ops.py silver` | `pipelines/batch/silver_hm.py` | Tipado, imputación, bandas de edad, validación con motivo de rechazo | `quality_report` con `conciliado = true` |
| A4 | Silver (BQ) | (incluido en A3) | `sql/silver/01_load_silver.sql` | `LOAD DATA OVERWRITE` desde Parquet; vista de eventos deduplicada | `retail_silver.articles/customers/transactions` |
| A5 | Gold | `ops.py gold` | `sql/gold/01_gold_model.sql`, `02_kpi_views.sql` | Modelo estrella, pedidos, popularidad, catálogo de servicio y vistas BI | `retail_gold.fact_sales` y `v_*` |
| A6 | Catálogo | `ops.py catalogo` | `api/load_catalog.py` | Cloud Run Job copia `v_catalog_serving` a Cloud SQL | `/ready` con 105,542 artículos |

## Reglas de calidad en Silver (motivo de rechazo)

| Motivo | Regla | Destino |
|---|---|---|
| `fecha_invalida` | `t_dat` no se puede convertir a fecha | `quarantine/transactions` |
| `sin_cliente` | `customer_id` nulo | `quarantine/transactions` |
| `articulo_inexistente` | `article_id` no existe en artículos | `quarantine/transactions` |
| `precio_invalido` | Precio nulo o ≤ 0 | `quarantine/transactions` |
| `canal_invalido` | `sales_channel_id` distinto de 1 o 2 | `quarantine/transactions` |

### Control de conciliación

El job falla si **entrada ≠ salida + rechazadas**. En la prueba local: 546,741 filas, 3 rechazos inyectados detectados, `conciliado = true`.

## 5.2 Pipeline B — Streaming (replay 2026)




**Figura 7.** *Dos rutas desde el mismo tópico: económica (suscripción BigQuery) y analítica (Dataflow).*

| Aspecto | Diseño |
|---|---|
| **Contrato del evento** | Campos obligatorios: `event_id`, `event_type`, `event_time`, `article_id`, `customer_id`, `price` |
| **Idempotencia** | `event_id` determinista; `silver.v_events` deduplica por `event_id` |
| **Ventanas** | Fijas de **60 s** con datos tardíos aceptados hasta **300 s** |
| **Métricas** | `retail_ops.stream_metrics_1m`: eventos, compras, importe, lag p50/p95 |
| **Errores** | Inválidos a `retail_ops.stream_dlq`; Pub/Sub reenvía al tópico DLQ tras 5 intentos |
| **Origen trazable** | `is_synthetic = true` y `source_t_dat` con la fecha original de H&M |
| **Costo** | Dataflow cobra mientras corre: `ops.py dataflow-detener` al terminar la demo |

## 5.3 Pipeline C — Servicio (API, asistente y tienda)

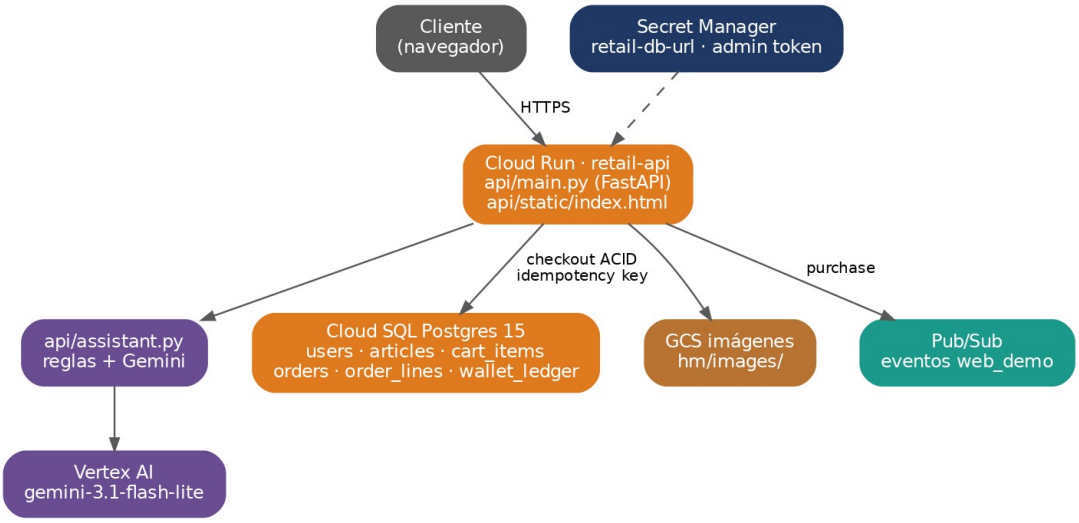



**Figura 8.** *Ruta de una compra: navegador → Cloud Run → Cloud SQL / Vertex AI → evento de vuelta a Pub/Sub.*

| Método | Ruta | Función |
|---|---|---|
| GET | /health, /ready | Salud del servicio y catálogo cargado |
| GET | /api/v1/catalog/products | Búsqueda en el catálogo (prefijo por tipo de producto) |
| GET | /api/v1/catalog/products/{article_id}/image | Imagen desde GCS o carpeta local |
| POST | /api/v1/recommendations | Recomendaciones por perfil; popularidad global para cold-start |
| POST | /api/v1/assistant/message | Asistente: Gemini interpreta, el backend busca en el catálogo real; respaldo por reglas |
| GET/POST/ DELETE | /api/v1/cart, /api/v1/cart/items | Carrito |
| POST | /api/v1/checkout | Compra ACID con idempotency key; **409** si no hay saldo |
| GET / POST | /api/v1/wallet, /api/v1/wallet/demo-topups | Saldo (libro append-only) y recarga con token de administrador |
| GET | /api/v1/orders, /api/v1/metrics/summary | Pedidos y métricas |

## 5.4 Pipeline D — Control de configuración y CI/CD

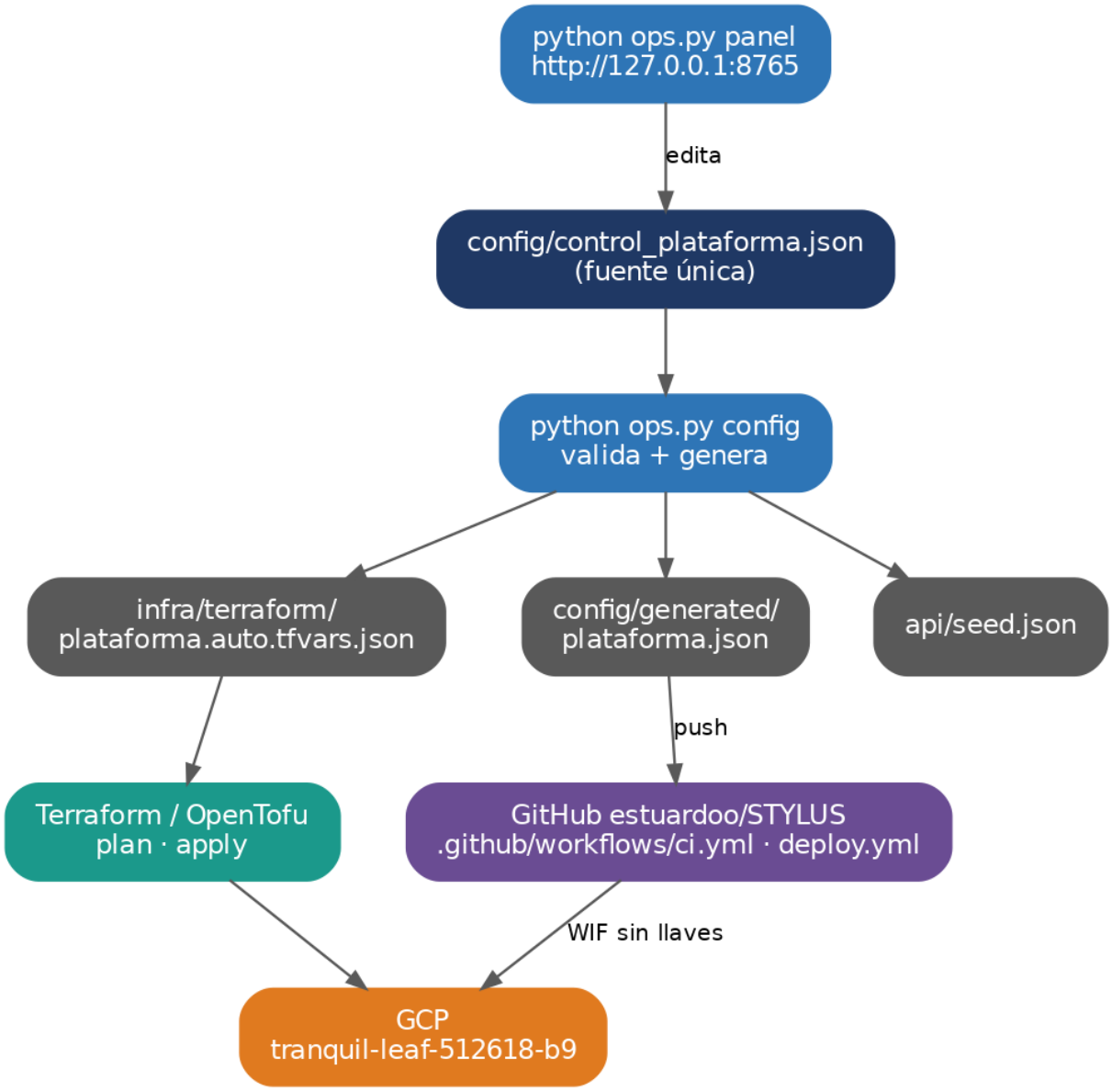



**Figura 9.** *Del JSON de control a GCP: validación, archivos generados, Terraform y GitHub Actions.*

| Workflow | Cuándo corre | Pasos |
|---|---|---|
| `ci.yml` | Cada push y pull request | Bloquea archivos prohibidos (CSV, Parquet, ZIP, `.db`, `.tfstate`, credenciales) · `ops.py config --check` · pruebas · calculadora de costos · `terraform fmt/validate` |
| `deploy.yml` | Push a `api/**` o `config/**`, o manual | Lee `config/generated/plataforma.json` · construye y publica la imagen · `gcloud run deploy` · prueba de humo. Se omite si no existe la variable `GCP_WIF_PROVIDER` |

# 6. Modelo de datos

Hay **30 tablas y vistas** en 5 zonas. El detalle de cada columna está en el **Anexo C** y el diagrama del modelo estrella en el **Anexo D**.

| Zona | Dataset / base | Propósito | Tablas y vistas |
|---|---|---|---|
| **BRONZE** | retail_bronze | Datos tal como llegan | ext_articles, ext_customers, ext_transactions, transactions, events_raw |
| **SILVER** | retail_silver | Datos limpios y tipados | articles, customers, transactions, v_events |
| **GOLD** | retail_gold | Modelo para negocio y BI | dim_date, dim_channel, dim_article, dim_customer, fact_sales, fact_orders, reco_popular, v_catalog_serving, v_ventas_mensuales, v_top_articulos, v_recurrencia_clientes |
| **OPS** | retail_ops | Operación y costos | stream_metrics_1m, stream_dlq, v_stream_salud, v_costo_real_mensual |
| **RETAIL** | Cloud SQL · retail | Transaccional de la tienda | users, articles, cart_items, orders, order_lines, wallet_ledger |

## 6.1 Modelo estrella (Gold)

**Grano de `fact_sales`**: una línea de venta (H&M registra una línea por unidad). `fact_orders` agrega por cliente + día + fuente porque el dataset no trae `order_id`. Las dimensiones de cliente, artículo, fecha y canal permiten responder ventas por segmento, categoría, temporada y canal.

| Tabla | Tipo | Grano | Partición / cluster | Origen |
|---|---|---|---|---|
| dim_date | TABLE | Calendario 2018-2027 | — | sql/gold/01_gold_model.sql |
| dim_channel | TABLE | Canales de venta (supuesto 1 tienda, 2 online) | — | sql/gold/01_gold_model.sql |
| dim_article | TABLE | Dimensión producto | article_id | sql/gold/01_gold_model.sql |
| dim_customer | TABLE | Dimensión cliente seudonimizado | customer_id | sql/gold/01_gold_model.sql |
| fact_sales | TABLE | Línea de venta: histórico H&M + eventos 2026 | sale_date / customer_id,article_id | sql/gold/01_gold_model.sql |
| fact_orders | TABLE | Pedido = cliente + día + fuente | order_date / customer_id | sql/gold/01_gold_model.sql |
| reco_popular | TABLE | Top por grupo, última semana | — | sql/gold/01_gold_model.sql |

> **Ver Anexo D**
> El diagrama completo del modelo estrella, con todas las columnas y sus tipos, está en el **Anexo D**; el diccionario columna por columna en el **Anexo C**.

## 6.2 Vistas para tableros

| Vista | Tablero | Pregunta de negocio |
|---|---|---|
| retail_gold.v_ventas_mensuales | Gerencial | ¿Cómo evolucionan ventas, pedidos y ticket promedio por mes y canal? |
| retail_gold.v_top_articulos | Gerencial | ¿Qué artículos y categorías concentran las ventas? |
| retail_gold.v_recurrencia_clientes | Gerencial | ¿Cuántos clientes recompran y con qué frecuencia? |
| retail_ops.v_stream_salud | Operativo | ¿El streaming está al día (eventos/min, lag p95, DLQ)? |
| retail_ops.v_costo_real_mensual | Costos | ¿Cuánto costó realmente el mes y cuánto por pedido? |

## 6.3 Recomendador

Evaluación con **split temporal real**: entrenamiento hasta 2020-09-16, prueba 2020-09-16 a 2020-09-22 (68,984 clientes, 26,447 en cold-start). El modelo B (recompra de 8 semanas + popularidad) **triplica** la precisión de la línea base.

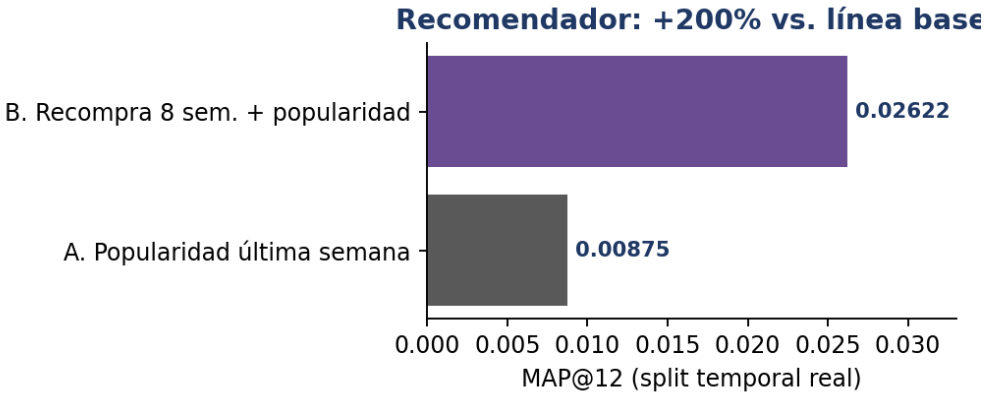



**Figura 10.** *MAP@12 de los modelos evaluados (pipelines/reco/baseline_eval.py).*

# 7. Configuración y operación

## 7.1 Fuente única: config/control_plataforma.json

Todos los insumos de la plataforma están en **un solo archivo versionado**. `python ops.py config` lo valida (formatos, roles que empiezan con `roles/`, referencias entre secciones, `roles/owner` prohibido por API) y genera de forma determinística los archivos que consumen Terraform, `ops.py` y la API.

| Sección | Contenido | Filas | Editable en el panel |
|---|---|---|---|
| parametros | Parámetros del proyecto | 64 | Sí (38 campos) |
| apis | APIs a activar | 21 | Sí |
| cuentas_servicio | Cuentas de servicio | 4 | Sí |
| permisos | Permisos IAM | 30 | Sí |
| usuarios | Equipo y usuarios de la tienda | 12 | Sí |
| datasets | Datasets BigQuery | 4 | Solo lectura |
| tablas | Tablas y vistas | 30 | Solo lectura |
| columnas | Columnas | 240 | Solo lectura |
| secretos | Secretos (solo nombres) | 2 | Solo lectura |

| Archivo generado | Lo consume | Contenido |
|---|---|---|
| infra/terraform/plataforma.auto.tfvars.json | Terraform / OpenTofu | Variables tipadas: buckets, datasets, tablas, IAM, Cloud SQL, Cloud Run, WIF |
| config/generated/plataforma.json | ops.py, GitHub Actions | Parámetros, nombres derivados, columnas, usuarios |
| api/seed.json | API (al iniciar) | Usuarios iniciales de la tienda y su saldo |

## 7.2 Panel de control web

`python ops.py panel` abre `http://127.0.0.1:8765` (solo en la máquina local). Tiene pestañas por sección del JSON, tablas editables con **agregar/quitar fila**, y un botón por cada comando de `ops.py` con el **registro en vivo** (Server-Sent Events). `apply` y `destroy` piden confirmación explícita.

## 7.3 Orquestador ops.py

| Comando | Qué hace | Requiere GCP |
|---|---|---|
| config [--check] | Valida el JSON y regenera los derivados | No |
| estado | Nombres y valores efectivos | No |
| local | Tienda en localhost:8080 con SQLite | No |
| panel | Panel web local | No |
| bootstrap | APIs, red, bucket de estado, terraform init | Sí |
| plan / apply / destroy | Ciclo de vida de la infraestructura | Sí |
| subir-datos | CSV a GCS con manifiesto | Sí |
| bronze / silver / gold | Capas del lakehouse | Sí |
| api / catalogo | Imagen, Cloud Run y catálogo en Cloud SQL | Sí |
| replay / dataflow-iniciar / dataflow-detener | Streaming | Sí |
| verificar | BigQuery real vs. JSON + /health | Sí |
| github | Valor de GCP_WIF_PROVIDER | Sí |
| costo-real | Vista de costo real por pedido | Sí |
| todo | bootstrap → apply → bronze → silver → gold → api → verificar | Sí |

## 7.4 Procedimiento de despliegue

| Paso | # | Acción | Comando | Responsable | Verificación |
|---|---|---|---|---|---|
| 1 | D01 | **Instalar herramientas** | Google Cloud CLI, Terraform >= 1.6, Python 3.10-3.12, git (o usar Cloud Shell, que ya las trae) | Winston Quispe | gcloud --version; terraform -version |
| 1 | D02 | **Crear proyecto y vincular facturación** | gcloud projects create <project_id>; gcloud billing projects link <project_id> --billing-account=<cuenta> | Winston Quispe | Proyecto con facturación en consola |
| 1 | D03 | **Completar el JSON de control** | Editar con python ops.py panel (o a mano) los campos marcados editables de config/control_plataforma.json: mínimo project_id y emails del equipo | Todo el equipo | python ops.py config sin errores |
| 1 | D04 | **Autenticarse** | gcloud auth login; gcloud auth application-default login (en Cloud Shell no hace falta) | Todo el equipo | gcloud config list |
| 1 | D05 | **Instalar dependencias** | pip install -r requirements.txt | Todo el equipo | python ops.py estado |
| 1 | D06 | **Publicar repo** | git remote add origin https://github.com/OWNER/REPO.git; git push -u origin main | Estuardo Oliver | Workflow ci en verde |
| 2 | D07 | **Preparar proyecto** | python ops.py bootstrap | Winston Quispe | APIs activas, bucket de estado, terraform init |
| 2 | D08 | **Revisar plan** | python ops.py plan | Winston Quispe | Plan sin errores; recursos esperados |
| 2 | D09 | **Crear infraestructura** | python ops.py apply | Winston Quispe | Buckets, datasets, tablas, Pub/Sub, IAM, Cloud SQL, Cloud Run |
| 2 | D10 | **Subir datos H&M (desde la PC con data/raw)** | python ops.py subir-datos (imágenes: --con-imagenes) | Nicole Arenas | gs://<bucket raw>/hm/*.csv |
| 2 | D11 | **Bronze** | python ops.py bronze | Nicole Arenas | 31,788,324 filas en bronze.transactions |
| 2 | D12 | **Silver (Serverless Spark)** | python ops.py silver | Nicole Arenas | _quality_report conciliado=true |
| 2 | D13 | **Gold y vistas BI** | python ops.py gold | Nicole Arenas | Tablas fact_/dim_ y vistas v_* |
| 2 | D14 | **Streaming de prueba** | python ops.py dataflow-iniciar; python ops.py replay; python ops.py dataflow-detener | Winston Quispe | Filas en ops.stream_metrics_1m y silver.v_events |
| 3 | D15 | **API, web y catálogo** | python ops.py api | Nicole Arenas | URL pública; /ready con 105,542 artículos |
| 3 | D16 | **Verificar contra el JSON de control** | python ops.py verificar | Todo el equipo | Matriz tabla por tabla: OK / FALTA / DIFERENTE |
| 3 | D17 | **Despliegue desde GitHub** | python ops.py github -> crear variable GCP_WIF_PROVIDER -> Actions > deploy-api | Winston Quispe | Workflow deploy-api en verde |
| 3 | D18 | **Tableros** | Looker Studio > Agregar datos > BigQuery > gold.v_* y ops.* | Gianmarco Mejia | Tablero gerencial y operativo |
| 3 | D19 | **Costos reales** | Consola: exportar facturación a BigQuery (dataset ops) -> parámetro billing_export_table -> python ops.py costo-real | Estuardo Oliver | ops.v_costo_real_mensual con datos |
| 3 | D20 | **Apagado** | python ops.py dataflow-detener; python ops.py destroy | Winston Quispe | Facturación diaria ~0 |

# 8. Seguridad, identidad y gobierno de datos

## 8.1 Cuentas de servicio (mínimo privilegio)

| Clave | Cuenta | Propósito | Roles asignados |
|---|---|---|---|
| **ingest** | sa-retail-ingest@tranquil-leaf-512618-b9.iam.gserviceaccount.com | Cargas a Bronze y publicación de eventos desde jobs | storage.objectAdmin, pubsub.publisher |
| **pipeline** | sa-retail-pipeline@tranquil-leaf-512618-b9.iam.gserviceaccount.com | Ejecuta Serverless Spark y Dataflow | dataproc.worker, dataflow.worker, bigquery.jobUser, storage.objectViewer, storage.objectAdmin, bigquery.dataEditor, bigquery.dataEditor, bigquery.dataEditor, pubsub.subscriber, pubsub.viewer |
| **api** | sa-retail-api@tranquil-leaf-512618-b9.iam.gserviceaccount.com | Identidad de Cloud Run (API, web y job de catálogo) | bigquery.jobUser, bigquery.dataViewer, aiplatform.user, cloudsql.client, secretmanager.secretAccessor, pubsub.publisher, storage.objectViewer |
| **deployer** | sa-retail-deployer@tranquil-leaf-512618-b9.iam.gserviceaccount.com | GitHub Actions publica imágenes y despliega Cloud Run | run.admin, artifactregistry.writer, iam.serviceAccountUser |

El detalle de los **30 permisos** (principal, rol, alcance y motivo) está en el **Anexo E**.

## 8.2 Secretos

| Secreto | Contenido | Lo genera | Lo consume |
|---|---|---|---|
| retail-db-url | URL de conexión a Cloud SQL con contraseña aleatoria | Terraform (random_password) | Cloud Run API y job de catálogo |
| retail-admin-token | Token para recargas demo (X-Admin-Token) | Terraform (random_password) | API /wallet/demo-topups |

## 8.3 Controles

| Control | Cómo se implementa | Evidencia |
|---|---|---|
| **Nada sensible en Git** | .gitignore (data/, .secrets/, *.tfstate, tokens) + regla en ci.yml | CI en verde; repositorio privado |
| **Secretos fuera del código** | Terraform genera contraseña y token en Secret Manager; Cloud Run los monta | cloudsql.tf, cloudrun.tf |
| **Despliegue sin llaves** | Workload Identity Federation para GitHub Actions | cicd.tf, deploy.yml |
| **Privilegio mínimo** | 4 cuentas de servicio con roles por recurso; roles/owner bloqueado en el JSON | Anexo E |
| **Datos seudonimizados** | customer_id es un hash en el dataset original | dim_customer |
| **Compras seguras** | Checkout ACID, idempotente, libro de saldo append-only (sin UPDATE/DELETE) | api/schema_pg.sql, pruebas |

> **Atención**
> La llave de cuenta de servicio de automatización (.secrets/gcp-admin-key.json) tiene rol **owner**: está fuera de Git, pero debe **rotarse o eliminarse** después del despliegue y reemplazarse por Workload Identity.

# 9. Costos y FinOps

`cost/assumptions.yaml` es la única fuente de supuestos; `python cost/estimate_costs.py` genera la calculadora con fórmulas (`docs/entrega1/Calculadora_Costos_GCP.xlsx`) y `--refresh-prices` actualiza las tarifas desde la API de Cloud Billing Catalog.

| Escenario | USD / mes | Pedidos / mes | USD / pedido |
|---|---|---|---|
| **Desarrollo** | USD 8.20 | 5,000 | USD 0.00164 |
| **Demo** | USD 11.57 | 20,000 | USD 0.00058 |
| **Producción** | USD 370.83 | 377,000 | USD 0.00098 |
| **Pico Black Friday** | USD 1,403.21 | 1,885,000 | USD 0.00074 |

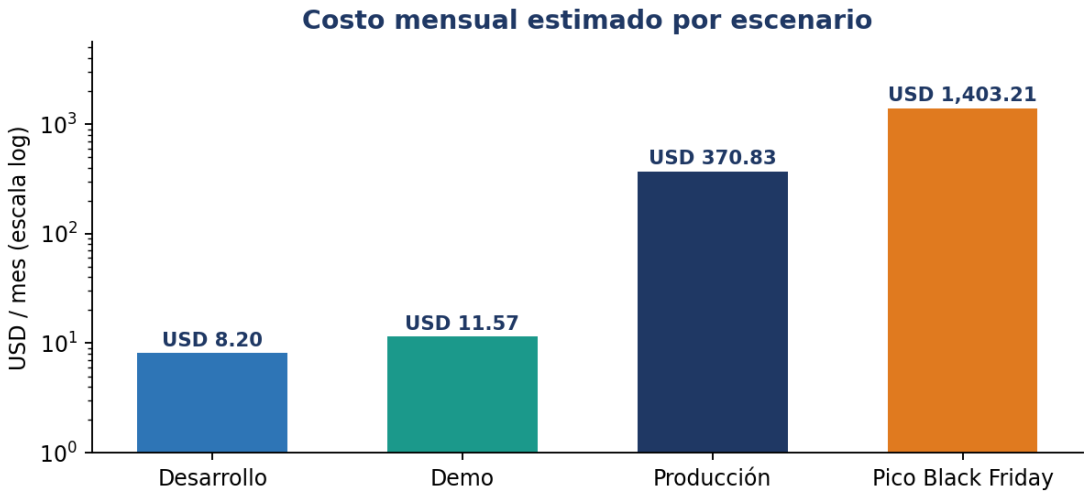



**Figura 11.** *Costo mensual por escenario (escala logarítmica).*

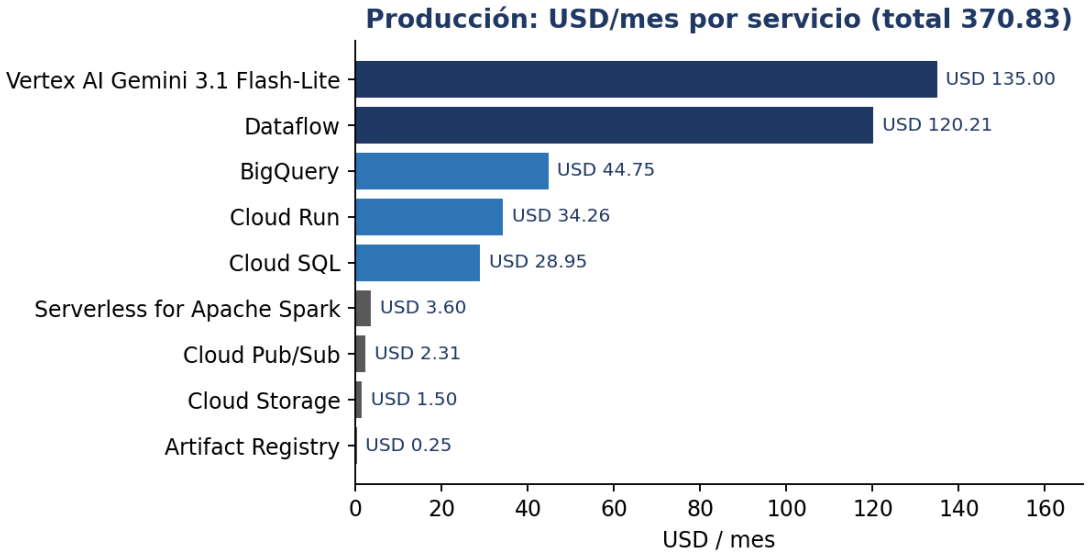



**Figura 12.** *Producción: distribución del costo por servicio.*

| Servicio | Desarrollo | Demo | Producción | Pico |
|---|---|---|---|---|
| **Vertex AI Gemini 3.1 Flash-Lite** | USD 1.50 | USD 2.75 | USD 135.00 | USD 675.00 |
| **Dataflow** | USD 0.00 | USD 3.95 | USD 120.21 | USD 360.64 |
| **BigQuery** | USD 0.04 | USD 0.10 | USD 44.75 | USD 120.15 |
| **Cloud Run** | USD 0.00 | USD 0.00 | USD 34.26 | USD 195.36 |
| **Cloud SQL** | USD 4.70 | USD 3.50 | USD 28.95 | USD 29.80 |
| **Serverless for Apache Spark** | USD 0.96 | USD 0.48 | USD 3.60 | USD 7.20 |
| **Cloud Pub/Sub** | USD 0.05 | USD 0.10 | USD 2.31 | USD 13.11 |
| **Cloud Storage** | USD 0.80 | USD 0.54 | USD 1.50 | USD 1.70 |
| **Artifact Registry** | USD 0.15 | USD 0.15 | USD 0.25 | USD 0.25 |

## 9.1 Palancas de ahorro

- **Gemini** y **Dataflow** explican el 69% del costo de producción: `assistant_mode = rules` y apagar Dataflow fuera de la demo son las dos palancas mayores.
- Cloud Run con **0 instancias mínimas**; Cloud SQL `db-f1-micro` en desarrollo.
- Imágenes a **Nearline** a los 30 días; consultas BigQuery sobre tablas **particionadas y agrupadas**.
- Presupuesto de **USD 50/mes** con alertas al 50/90/100% (Terraform) y costo real por pedido (`cost/real_cost_per_order.sql`).

# 10. Calidad, estado y hoja de ruta

## 10.1 Pruebas automatizadas

| Archivo | Prueba | Qué garantiza |
|---|---|---|
| tests/test_config.py | test_generados_al_dia | Los archivos generados coinciden con el JSON de control |
| tests/test_config.py | test_validacion_detecta_errores | Un permiso inválido o un principal inexistente se rechaza |
| tests/test_config.py | test_contrato_cloudsql_igual_a_schema_sql | Las columnas de Cloud SQL del JSON son las de api/schema.sql |
| tests/test_api.py | test_flujo_compra_segura | Checkout idempotente, sin saldo negativo, 409 sin fondos |
| tests/test_api.py | test_reco_cold_start_y_health | Recomendaciones para clientes nuevos y salud del servicio |
| tests/test_core.py | test_event_2026_preserva_origen_e_idempotente | El replay conserva la fecha original y no duplica |
| tests/test_core.py | test_free_tier_y_total | La calculadora aplica el nivel gratuito y suma bien |

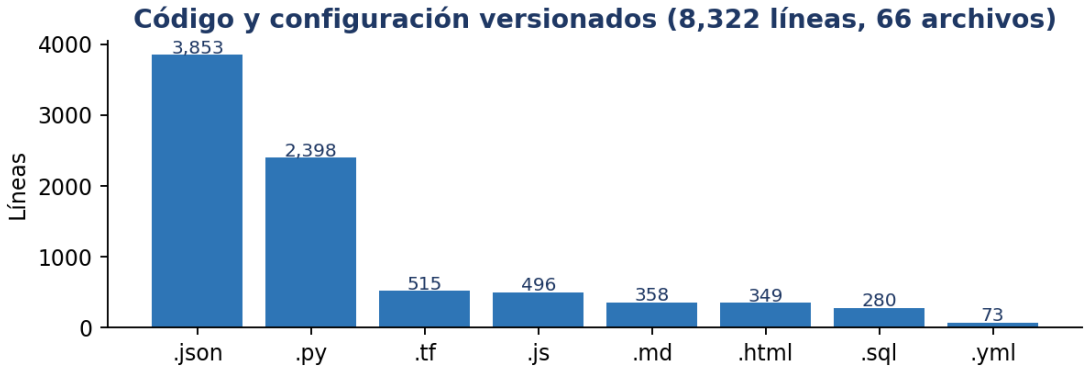



**Figura 13.** *Código y configuración versionados por tipo de archivo.*

## 10.2 Estado por entrega

### Paso 1 — 81% hecho

| ID | Tarea | Responsable | Estado | Evidencia |
|---|---|---|---|---|
| P1-01 | Auditoría de datos H&M | Nicole Arenas | ✔ Hecho | 31.8M líneas, 9.08M pedidos, 1.37M clientes, 105,542 artículos |
| P1-02 | Ordenar carpeta y repositorio | Estuardo Oliver | ✔ Hecho | data/_archive y .secrets/ fuera de Git |
| P1-03 | Proteger datos y secretos | Winston Quispe | ✔ Hecho | Regla 'Sin datos ni secretos versionados' |
| P1-04 | Justificar stack GCP | Winston Quispe | ✔ Hecho | Tabla de ADR |
| P1-05 | Diagrama end-to-end | Gianmarco Mejia | ✔ Hecho | Graphviz regenerable |
| P1-06 | Integración multifuente | Nicole Arenas | ✔ Hecho | — |
| P1-07 | Ingesta y almacenamiento | Nicole Arenas | ✔ Hecho | — |
| P1-08 | Escalabilidad Black Friday | Winston Quispe | ✔ Hecho | Escenario 5x en calculadora |
| P1-09 | Calculadora y costo por pedido | Gianmarco Mejia | ✔ Hecho | Año 1 USD 5,482; USD 0.00098/pedido (Gemini 3.1 Flash-Lite) |
| P1-10 | Automatizar control de costos | Winston Quispe | ● Listo p/desplegar | Probado local; activar billing export en consola |
| P1-11 | DRP / HA | Winston Quispe | ✔ Hecho | — |
| P1-12 | Catálogo de KPIs | Gianmarco Mejia | ✔ Hecho | — |
| P1-13 | Ingesta Bronze con manifiesto | Nicole Arenas | ✔ Hecho | Manifiesto real en data/manifests/ |
| P1-14 | Terraform base | Winston Quispe | ● Listo p/desplegar | validate OK; tipos de variables probados con el JSON generado |
| P1-15 | Documento Word Entrega 1 | Estuardo Oliver | ✔ Hecho | — |

| ID | Tarea | Responsable | Estado | Evidencia |
|---|---|---|---|---|
| P1-16 | Matriz de auditoría y roles | Estuardo Oliver | ✔ Hecho | Este archivo |
| P1-17 | Crear proyecto GCP | Winston Quispe | ✔ Hecho | Proyecto tranquil-leaf-512618-b9 con facturación vinculada; cuenta de servicio con rol owner |
| P1-18 | Publicar repositorio GitHub | Winston Quispe | ✔ Hecho | github.com/estuardoo/STYLUS (privado); workflow ci en verde |
| P1-20 | JSON de control de la plataforma | Winston Quispe | ✔ Hecho | 30 tablas, 240 columnas, 28 permisos, 6 usuarios app; CI verifica generados |
| P1-21 | Orquestador de despliegue | Winston Quispe | ● Listo p/desplegar | config/estado/local probados; comandos de nube pendientes de GCP |
| P1-19 | Revisión y envío Entrega 1 | Estuardo Oliver | ✖ Pendiente | — |

## Paso 2 — 14% hecho

| ID | Tarea | Responsable | Estado | Evidencia |
|---|---|---|---|---|
| P2-01 | Desplegar infraestructura dev | Winston Quispe | ✖ Pendiente | — |
| P2-02 | Cargar Bronze (GCS + BigQuery) | Nicole Arenas | ✖ Pendiente | SQL y script listos |
| P2-03 | Silver con Spark | Nicole Arenas | ● Listo p/desplegar | Local: 546,741 filas, 3 rechazos inyectados detectados, conciliado; corregido fn/active |
| P2-04 | Silver en BigQuery + vista de eventos | Nicole Arenas | ● Listo p/desplegar | — |
| P2-05 | Gold: modelo estrella | Nicole Arenas | ● Listo p/desplegar | — |
| P2-06 | Replay de eventos 2026 | Winston Quispe | ● Listo p/desplegar | Dry-run sobre datos reales OK |
| P2-07 | Ruta económica Pub/Sub -> BigQuery | Winston Quispe | ● Listo p/desplegar | — |
| P2-08 | Pipeline Dataflow | Winston Quispe | ● Listo p/desplegar | DirectRunner: 350 mensajes -> 300 únicos, 2 a DLQ |
| P2-09 | Popularidad y catálogo de servicio | Estuardo Oliver | ● Listo p/desplegar | — |
| P2-10 | Recomendador baseline + evaluación | Estuardo Oliver | ✔ Hecho | MAP@12 0.00875 vs 0.02622 (+200%) |
| P2-11 | Diccionario de datos | Nicole Arenas | ✔ Hecho | — |
| P2-12 | Vistas para tableros | Gianmarco Mejia | ● Listo p/desplegar | — |
| P2-13 | Dashboard borrador | Gianmarco Mejia | ✖ Pendiente | — |
| P2-14 | Informe Entrega 2 | Estuardo Oliver | ✖ Pendiente | — |

## Paso 3 — 6% hecho

| ID | Tarea | Responsable | Estado | Evidencia |
|---|---|---|---|---|
| P3-01 | API FastAPI | Nicole Arenas | ● Listo p/desplegar | Pruebas automáticas OK |
| P3-02 | Wallet transaccional | Nicole Arenas | ● Listo p/desplegar | Test: doble clic = 1 cobro; 409 sin saldo |
| P3-03 | Asistente Vertex AI Gemini | Estuardo Oliver | ● Listo p/desplegar | Reglas probadas con catálogo real; Gemini requiere GCP |
| P3-04 | Recomendaciones en API | Estuardo Oliver | ● Listo p/desplegar | — |
| P3-05 | Web tienda | Nicole Arenas | ● Listo p/desplegar | — |
| P3-06 | Cloud SQL + Cloud Run + secretos | Winston Quispe | ● Listo p/desplegar | tofu validate OK |
| P3-07 | CI/CD sin llaves | Winston Quispe | ● Listo p/desplegar | — |
| P3-08 | Catálogo Gold -> Cloud SQL | Nicole Arenas | ✖ Pendiente | Local: 105,542 artículos cargados |
| P3-09 | Dashboard gerencial | Gianmarco Mejia | ✖ Pendiente | — |
| P3-10 | Dashboard operativo y de costos | Gianmarco Mejia | ✖ Pendiente | — |
| P3-11 | Prueba de recuperación | Winston Quispe | ✖ Pendiente | Runbook listo |
| P3-12 | Costo real vs estimado | Gianmarco Mejia | ✖ Pendiente | — |
| P3-13 | Guion de demo end-to-end | Estuardo Oliver | ✔ Hecho | — |
| P3-14 | Slides (8 puntos del PDF) | Gianmarco Mejia | ✖ Pendiente | — |
| P3-15 | Documento final y trazabilidad | Estuardo Oliver | ✖ Pendiente | — |
| P3-16 | Apagado post-demo | Winston Quispe | ✖ Pendiente | — |

## 10.3 Riesgos y mitigación

| Riesgo | Prob. | Impacto | Mitigación | Dueño |
|---|---|---|---|---|
| Infraestructura real aún no desplegada (bloquea 15 tareas listas) | Alta | Alta | Ejecutar `ops.py todo` desde Cloud Shell esta semana | Winston Quispe |
| Dataflow encendido fuera de la demo | Media | Alta | `dataflow-detener` en el checklist; alertas de presupuesto | Winston Quispe |
| Llave owner en disco local | Media | Alta | Rotar/eliminar tras el despliegue; usar WIF | Estuardo Oliver |
| Retiro de modelos Gemini (ya ocurrió con 2.5) | Media | Media | Modelo como parámetro + modo `rules` de respaldo | Estuardo Oliver |
| Emails del equipo vacíos (sin acceso IAM) | Alta | Media | Completar la sección `usuarios` en el panel | Todo el equipo |
| Tableros sin construir | Media | Media | Vistas `v_*` ya definidas; conectar Looker Studio tras Gold | Gianmarco Mejia |
| Pedido inferido (sin `order_id`) | Alta | Baja | Definición documentada cliente + día + fuente | Nicole Arenas |

## 10.4 Próximos pasos

1. **Hoy (24-oct)**: revisión cruzada y envío de la Entrega 1 — Estuardo Oliver.
2. **Semana 1**: `bootstrap`, `apply`, `subir-datos`, `bronze` en `tranquil-leaf-512618-b9` — Winston Quispe y Nicole Arenas.
3. **Semana 2**: `silver`, `gold`, `replay` + Dataflow, primer tablero, informe de Entrega 2 (07-nov).
4. **Semanas 3-4**: `api`, catálogo en Cloud SQL, tableros finales, prueba de DRP, costo real, slides y sustentación (20/21-nov).

# Anexo A. Inventario de rutas del repositorio

Archivos versionados en `github.com/estuardoo/STYLUS` (rama `main`). Total: **66 archivos**. Los datos (`data/`) y secretos (`.secrets/`) existen solo en la PC del equipo y están excluidos de Git.

| Ruta | Líneas | Propósito |
|---|---|---|
| .github/workflows/ci.yml | 29 | CI/CD (GitHub Actions) |
| .github/workflows/deploy.yml | 44 | CI/CD (GitHub Actions) |
| .gitignore | 47 | Exclusión de datos y secretos |
| README.md | 188 | Portada del repositorio |
| api/.dockerignore | 2 | API FastAPI, asistente y carga de catálogo |
| api/Dockerfile | 7 | API FastAPI, asistente y carga de catálogo |
| api/assistant.py | 98 | API FastAPI, asistente y carga de catálogo |
| api/load_catalog.py | 88 | API FastAPI, asistente y carga de catálogo |
| api/main.py | 312 | API FastAPI, asistente y carga de catálogo |
| api/requirements.txt | 8 | API FastAPI, asistente y carga de catálogo |
| api/schema.sql | 55 | API FastAPI, asistente y carga de catálogo |
| api/schema_pg.sql | 3 | API FastAPI, asistente y carga de catálogo |
| api/seed.json | 34 | Usuarios iniciales (generado) |
| api/static/index.html | 145 | Tienda web |
| config/control_plataforma.json | 3533 | Fuente única de configuración |
| config/generated/plataforma.json | 3114 | Generado por ops.py config (no editar) |
| cost/assumptions.yaml | 58 | Calculadora y costo real |
| cost/estimate_costs.py | 214 | Calculadora y costo real |
| cost/real_cost_per_order.sql | 23 | Calculadora y costo real |
| docs/DEPLOY.md | 53 | Documentación |
| docs/_generadores/LEEME.md | 14 | Generadores de documentos |
| docs/_generadores/build_matriz.py | 234 | Generadores de documentos |
| docs/_generadores/build_word.js | 496 | Generadores de documentos |
| docs/_generadores/toc_embed.py | 18 | Generadores de documentos |
| docs/_generadores/toc_pages.py | 10 | Generadores de documentos |
| docs/architecture/arquitectura.png | 2575 | Diagramas de arquitectura |
| docs/architecture/arquitectura_horizontal.dot | 59 | Diagramas de arquitectura |
| docs/architecture/arquitectura_horizontal.png | 2008 | Diagramas de arquitectura |
| docs/architecture/arquitectura_vertical.dot | 59 | Diagramas de arquitectura |
| docs/demo_script.md | 19 | Documentación |
| docs/diccionario_datos.md | 51 | Documentación |
| docs/entrega1/Calculadora_Costos_GCP.xlsx | 113 | Entregables de la Entrega 1 |
| docs/entrega1/Entrega1_Arquitectura_BigData_Retail.docx | 3006 | Entregables de la Entrega 1 |
| docs/entrega1/Entrega1_Arquitectura_BigData_Retail.pdf | 20184 | Entregables de la Entrega 1 |
| docs/entrega1/Guia_Tecnica_STYLUS.docx | 8849 | Entregables de la Entrega 1 |
| docs/entrega1/Guia_Tecnica_STYLUS.pdf | 8148 | Entregables de la Entrega 1 |
| docs/evaluacion_recomendador.json | 12 | Documentación |
| docs/evaluacion_recomendador.md | 12 | Documentación |
| docs/gestion/Matriz_Auditoria_Proyecto.xlsx | 160 | Matriz de auditoría |
| docs/referencia/Proyecto.pdf | 1346 | Documentación |
| docs/referencia/v1_prompt_blueprint/Blueprint_y_Prompt_Maestro_Retail_BigData.docx | 482 | Documentación |
| docs/referencia/v1_prompt_blueprint/prompt_maestro_retail_bigdata.txt | 169 | Documentación |
| docs/runbooks/drp.md | 21 | Runbooks de operación / DRP |
| infra/terraform/cicd.tf | 28 | Infraestructura como código |
| infra/terraform/cloudrun.tf | 76 | Infraestructura como código |
| infra/terraform/cloudsql.tf | 83 | Infraestructura como código |

| Ruta | Líneas | Propósito |
|---|---|---|
| infra/terraform/main.tf | 254 | Infraestructura como código |
| infra/terraform/outputs.tf | 9 | Infraestructura como código |
| infra/terraform/plataforma.auto.tfvars.json | 308 | Variables de Terraform generadas |
| infra/terraform/variables.tf | 65 | Infraestructura como código |
| ingestion/batch/upload_raw.py | 87 | Carga de CSV a GCS |
| ingestion/streaming/replay_publisher.py | 93 | Replay de eventos 2026 |
| ops.py | 624 | Orquestador de la plataforma |
| panel/app.py | 186 | Panel web local de configuración y despliegue |
| panel/static/index.html | 204 | Panel web local de configuración y despliegue |
| pipelines/batch/silver_hm.py | 112 | Spark: Bronze → Silver |
| pipelines/reco/baseline_eval.py | 74 | Evaluación del recomendador |
| pipelines/streaming/dataflow_events.py | 116 | Dataflow / Apache Beam |
| requirements.txt | 11 | Dependencias Python |
| sql/bronze/01_bronze_hm.sql | 58 | SQL de la capa Bronze |
| sql/gold/01_gold_model.sql | 72 | SQL del modelo estrella y vistas BI |
| sql/gold/02_kpi_views.sql | 34 | SQL del modelo estrella y vistas BI |
| sql/silver/01_load_silver.sql | 35 | SQL de la capa Silver |
| tests/test_api.py | 60 | Pruebas unitarias |
| tests/test_config.py | 40 | Pruebas unitarias |
| tests/test_core.py | 32 | Pruebas unitarias |

# Anexo B. Catálogo de tablas y vistas

| Motor | Dataset real | Tabla | Tipo | Creado por | Partición / cluster | Descripción |
|---|---|---|---|---|---|---|
| bigquery | retail_bronze | **ext_articles** | EXTERNAL | sql/bronze/01_bronze_hm.sql | — | CSV articles.csv en GCS (todo texto) |
| bigquery | retail_bronze | **ext_customers** | EXTERNAL | sql/bronze/01_bronze_hm.sql | — | CSV customers.csv en GCS (todo texto) |
| bigquery | retail_bronze | **ext_transactions** | EXTERNAL | sql/bronze/01_bronze_hm.sql | — | CSV transactions_train.csv en GCS |
| bigquery | retail_bronze | **transactions** | TABLE | sql/bronze/01_bronze_hm.sql | t_dat / customer_id,article_id | Copia nativa particionada de las transacciones |
| bigquery | retail_bronze | **events_raw** | TABLE | terraform | publish_time | Eventos tal como llegan por la suscripción Pub/Sub -> BigQuery |
| bigquery | retail_silver | **articles** | TABLE | pipelines/batch/silver_hm.py + sql/silver/01_load_silver.sql | — | Artículos tipados; detail_desc imputada |
| bigquery | retail_silver | **customers** | TABLE | pipelines/batch/silver_hm.py + sql/silver/01_load_silver.sql | customer_id | Clientes con nulos tratados y banda de edad |
| bigquery | retail_silver | **transactions** | TABLE | pipelines/batch/silver_hm.py + sql/silver/01_load_silver.sql | t_dat / customer_id,article_id | Transacciones válidas (rechazos en quarantine/) |
| bigquery | retail_silver | **v_events** | VIEW | sql/silver/01_load_silver.sql | — | Eventos de events_raw deduplicados por event_id |
| bigquery | retail_gold | **dim_date** | TABLE | sql/gold/01_gold_model.sql | — | Calendario 2018-2027 |
| bigquery | retail_gold | **dim_channel** | TABLE | sql/gold/01_gold_model.sql | — | Canales de venta (supuesto 1 tienda, 2 online) |
| bigquery | retail_gold | **dim_article** | TABLE | sql/gold/01_gold_model.sql | article_id | Dimensión producto |
| bigquery | retail_gold | **dim_customer** | TABLE | sql/gold/01_gold_model.sql | customer_id | Dimensión cliente seudonimizado |
| bigquery | retail_gold | **fact_sales** | TABLE | sql/gold/01_gold_model.sql | sale_date / customer_id,article_id | Línea de venta: histórico H&M + eventos 2026 |
| bigquery | retail_gold | **fact_orders** | TABLE | sql/gold/01_gold_model.sql | order_date / customer_id | Pedido = cliente + día + fuente |
| bigquery | retail_gold | **reco_popular** | TABLE | sql/gold/01_gold_model.sql | — | Top por grupo, última semana |
| bigquery | retail_gold | **v_catalog_serving** | VIEW | sql/gold/01_gold_model.sql | — | Catálogo que se copia a Cloud SQL |
| bigquery | retail_gold | **v_ventas_mensuales** | VIEW | sql/gold/02_kpi_views.sql | — | Tablero gerencial |
| bigquery | retail_gold | **v_top_articulos** | VIEW | sql/gold/02_kpi_views.sql | — | Tablero gerencial |
| bigquery | retail_gold | **v_recurrencia_clientes** | VIEW | sql/gold/02_kpi_views.sql | — | Tablero gerencial |
| bigquery | retail_ops | **stream_metrics_1m** | TABLE | terraform | window_start | Métricas de Dataflow por minuto |
| bigquery | retail_ops | **stream_dlq** | TABLE | terraform | recibido | Mensajes inválidos rechazados por Dataflow |
| bigquery | retail_ops | **v_stream_salud** | VIEW | sql/gold/02_kpi_views.sql | — | Tablero operativo |
| bigquery | retail_ops | **v_costo_real_mensual** | VIEW | cost/real_cost_per_order.sql (opcional) | — | Costo facturado y USD por pedido |
| cloudsql | Cloud SQL · retail | **users** | TABLE | api/schema.sql | — | Usuarios de la tienda demo (semilla: hoja Usuarios tipo app) |
| cloudsql | Cloud SQL · retail | **articles** | TABLE | api/schema.sql | — | Catálogo de servicio (copia de gold.v_catalog_serving) |
| cloudsql | Cloud SQL · retail | **cart_items** | TABLE | api/schema.sql | — | Carrito |
| cloudsql | Cloud SQL · retail | **orders** | TABLE | api/schema.sql | — | Pedidos |
| cloudsql | Cloud SQL · retail | **order_lines** | TABLE | api/schema.sql | — | Líneas de pedido |
| cloudsql | Cloud SQL · retail | **wallet_ledger** | TABLE | api/schema.sql | — | Movimientos de saldo append-only (saldo = suma) |

# Anexo C. Diccionario de columnas

**240 columnas** en 30 tablas, agrupadas por tabla. Fuente: sección `columnas` del JSON de control; `ops.py verificar` las compara contra BigQuery real.

### retail_bronze.ext_articles (bigquery, 25 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| article_id | STRING | NULLABLE |
| product_code | STRING | NULLABLE |
| prod_name | STRING | NULLABLE |
| product_type_no | STRING | NULLABLE |
| product_type_name | STRING | NULLABLE |
| product_group_name | STRING | NULLABLE |
| graphical_appearance_no | STRING | NULLABLE |
| graphical_appearance_name | STRING | NULLABLE |
| colour_group_code | STRING | NULLABLE |
| colour_group_name | STRING | NULLABLE |
| perceived_colour_value_id | STRING | NULLABLE |
| perceived_colour_value_name | STRING | NULLABLE |
| perceived_colour_master_id | STRING | NULLABLE |
| perceived_colour_master_name | STRING | NULLABLE |
| department_no | STRING | NULLABLE |
| department_name | STRING | NULLABLE |
| index_code | STRING | NULLABLE |
| index_name | STRING | NULLABLE |
| index_group_no | STRING | NULLABLE |
| index_group_name | STRING | NULLABLE |
| section_no | STRING | NULLABLE |
| section_name | STRING | NULLABLE |
| garment_group_no | STRING | NULLABLE |
| garment_group_name | STRING | NULLABLE |
| detail_desc | STRING | NULLABLE |

### retail_bronze.ext_customers (bigquery, 7 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| customer_id | STRING | NULLABLE |
| FN | STRING | NULLABLE |
| Active | STRING | NULLABLE |
| club_member_status | STRING | NULLABLE |
| fashion_news_frequency | STRING | NULLABLE |
| age | STRING | NULLABLE |
| postal_code | STRING | NULLABLE |

### retail_bronze.ext_transactions (bigquery, 5 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| t_dat | DATE | NULLABLE |
| customer_id | STRING | NULLABLE |
| article_id | STRING | NULLABLE |
| price | FLOAT64 | NULLABLE |
| sales_channel_id | INT64 | NULLABLE |

### retail_bronze.transactions (bigquery, 7 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| **t_dat** | DATE | NULLABLE |
| **customer_id** | STRING | NULLABLE |
| **article_id** | STRING | NULLABLE |
| **price** | FLOAT64 | NULLABLE |
| **sales_channel_id** | INT64 | NULLABLE |
| **_ingested_at** | TIMESTAMP | NULLABLE |
| **_source_system** | STRING | NULLABLE |

### retail_bronze.events_raw (bigquery, 5 columnas)

| Columna | Tipo | Modo | Descripción |
|---|---|---|---|
| **subscription_name** | STRING | NULLABLE | — |
| **message_id** | STRING | NULLABLE | — |
| **publish_time** | TIMESTAMP | NULLABLE | — |
| **data** | STRING | NULLABLE | JSON del evento |
| **attributes** | STRING | NULLABLE | — |

### retail_silver.articles (bigquery, 26 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| **article_id** | STRING | NULLABLE |
| **product_code** | STRING | NULLABLE |
| **prod_name** | STRING | NULLABLE |
| **product_type_no** | INT64 | NULLABLE |
| **product_type_name** | STRING | NULLABLE |
| **product_group_name** | STRING | NULLABLE |
| **graphical_appearance_no** | INT64 | NULLABLE |
| **graphical_appearance_name** | STRING | NULLABLE |
| **colour_group_code** | INT64 | NULLABLE |
| **colour_group_name** | STRING | NULLABLE |
| **perceived_colour_value_id** | INT64 | NULLABLE |
| **perceived_colour_value_name** | STRING | NULLABLE |
| **perceived_colour_master_id** | INT64 | NULLABLE |
| **perceived_colour_master_name** | STRING | NULLABLE |
| **department_no** | INT64 | NULLABLE |
| **department_name** | STRING | NULLABLE |
| **index_code** | STRING | NULLABLE |
| **index_name** | STRING | NULLABLE |
| **index_group_no** | INT64 | NULLABLE |
| **index_group_name** | STRING | NULLABLE |
| **section_no** | INT64 | NULLABLE |
| **section_name** | STRING | NULLABLE |
| **garment_group_no** | INT64 | NULLABLE |
| **garment_group_name** | STRING | NULLABLE |
| **detail_desc** | STRING | NULLABLE |
| **_ingested_at** | TIMESTAMP | NULLABLE |

### retail_silver.customers (bigquery, 9 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| customer_id | STRING | NULLABLE |
| club_member_status | STRING | NULLABLE |
| fashion_news_frequency | STRING | NULLABLE |
| age | INT64 | NULLABLE |
| postal_code | STRING | NULLABLE |
| fn | INT64 | NULLABLE |
| active | INT64 | NULLABLE |
| age_band | STRING | NULLABLE |
| _ingested_at | TIMESTAMP | NULLABLE |

### retail_silver.transactions (bigquery, 7 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| article_id | STRING | NULLABLE |
| t_dat | DATE | NULLABLE |
| customer_id | STRING | NULLABLE |
| price | FLOAT64 | NULLABLE |
| sales_channel_id | INT64 | NULLABLE |
| _source_system | STRING | NULLABLE |
| _ingested_at | TIMESTAMP | NULLABLE |

### retail_silver.v_events (bigquery, 14 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| event_id | STRING | NULLABLE |
| event_type | STRING | NULLABLE |
| event_time | TIMESTAMP | NULLABLE |
| source_t_dat | DATE | NULLABLE |
| customer_id | STRING | NULLABLE |
| article_id | STRING | NULLABLE |
| price | FLOAT64 | NULLABLE |
| qty | INT64 | NULLABLE |
| sales_channel_id | INT64 | NULLABLE |
| order_id | STRING | NULLABLE |
| source_system | STRING | NULLABLE |
| is_synthetic | BOOL | NULLABLE |
| simulation_run_id | STRING | NULLABLE |
| publish_time | TIMESTAMP | NULLABLE |

### retail_gold.dim_date (bigquery, 7 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| date_key | DATE | NULLABLE |
| anio | INT64 | NULLABLE |
| mes | INT64 | NULLABLE |
| anio_mes | STRING | NULLABLE |
| semana_iso | INT64 | NULLABLE |
| dia_semana | STRING | NULLABLE |
| fin_de_semana | BOOL | NULLABLE |

## retail_gold.dim_channel (bigquery, 3 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| sales_channel_id | INT64 | NULLABLE |
| canal | STRING | NULLABLE |
| nota | STRING | NULLABLE |

## retail_gold.dim_article (bigquery, 13 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| article_id | STRING | NULLABLE |
| product_code | STRING | NULLABLE |
| prod_name | STRING | NULLABLE |
| product_type_name | STRING | NULLABLE |
| product_group_name | STRING | NULLABLE |
| colour_group_name | STRING | NULLABLE |
| perceived_colour_master_name | STRING | NULLABLE |
| department_name | STRING | NULLABLE |
| index_name | STRING | NULLABLE |
| index_group_name | STRING | NULLABLE |
| section_name | STRING | NULLABLE |
| garment_group_name | STRING | NULLABLE |
| detail_desc | STRING | NULLABLE |

## retail_gold.dim_customer (bigquery, 7 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| customer_id | STRING | NULLABLE |
| age | INT64 | NULLABLE |
| age_band | STRING | NULLABLE |
| club_member_status | STRING | NULLABLE |
| fashion_news_frequency | STRING | NULLABLE |
| fn | INT64 | NULLABLE |
| active | INT64 | NULLABLE |

## retail_gold.fact_sales (bigquery, 9 columnas)

| Columna | Tipo | Modo | Descripción |
|---|---|---|---|
| sale_date | DATE | NULLABLE | — |
| customer_id | STRING | NULLABLE | — |
| article_id | STRING | NULLABLE | — |
| sales_channel_id | INT64 | NULLABLE | — |
| qty | INT64 | NULLABLE | — |
| price | FLOAT64 | NULLABLE | — |
| source | STRING | NULLABLE | hm_hist \| hm_transactions_replay \| web_demo |
| is_synthetic | BOOL | NULLABLE | — |
| order_id | STRING | NULLABLE | — |

### retail_gold.fact_orders (bigquery, 9 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| order_id | STRING | NULLABLE |
| order_date | DATE | NULLABLE |
| customer_id | STRING | NULLABLE |
| sales_channel_id | INT64 | NULLABLE |
| source | STRING | NULLABLE |
| is_synthetic | BOOL | NULLABLE |
| lineas | INT64 | NULLABLE |
| unidades | INT64 | NULLABLE |
| importe | FLOAT64 | NULLABLE |

### retail_gold.reco_popular (bigquery, 4 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| index_group_name | STRING | NULLABLE |
| article_id | STRING | NULLABLE |
| ventas_7d | INT64 | NULLABLE |
| rank_grupo | INT64 | NULLABLE |

### retail_gold.v_catalog_serving (bigquery, 10 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| article_id | STRING | NULLABLE |
| prod_name | STRING | NULLABLE |
| product_type | STRING | NULLABLE |
| product_group | STRING | NULLABLE |
| index_group | STRING | NULLABLE |
| section | STRING | NULLABLE |
| colour | STRING | NULLABLE |
| detail_desc | STRING | NULLABLE |
| price_cents | INT64 | NULLABLE |
| popularity | INT64 | NULLABLE |

### retail_gold.v_ventas_mensuales (bigquery, 8 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| anio_mes | STRING | NULLABLE |
| canal | STRING | NULLABLE |
| source | STRING | NULLABLE |
| is_synthetic | BOOL | NULLABLE |
| pedidos | INT64 | NULLABLE |
| unidades | INT64 | NULLABLE |
| importe | FLOAT64 | NULLABLE |
| ticket_medio | FLOAT64 | NULLABLE |

### retail_gold.v_top_articulos (bigquery, 7 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| anio_mes | STRING | NULLABLE |
| index_group_name | STRING | NULLABLE |
| product_type_name | STRING | NULLABLE |
| prod_name | STRING | NULLABLE |
| article_id | STRING | NULLABLE |
| lineas | INT64 | NULLABLE |
| importe | FLOAT64 | NULLABLE |

### retail_gold.v_recurrencia_clientes (bigquery, 4 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| age_band | STRING | NULLABLE |
| clientes | INT64 | NULLABLE |
| pedidos | INT64 | NULLABLE |
| pedidos_por_cliente | FLOAT64 | NULLABLE |

### retail_ops.stream_metrics_1m (bigquery, 8 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| window_start | TIMESTAMP | NULLABLE |
| window_end | TIMESTAMP | NULLABLE |
| eventos | INT64 | NULLABLE |
| compras | INT64 | NULLABLE |
| importe | FLOAT64 | NULLABLE |
| lag_p50_s | FLOAT64 | NULLABLE |
| lag_p95_s | FLOAT64 | NULLABLE |
| emitido | TIMESTAMP | NULLABLE |

### retail_ops.stream_dlq (bigquery, 3 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| recibido | TIMESTAMP | NULLABLE |
| error | STRING | NULLABLE |
| payload | STRING | NULLABLE |

### retail_ops.v_stream_salud (bigquery, 5 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| minuto | TIMESTAMP | NULLABLE |
| source_system | STRING | NULLABLE |
| eventos | INT64 | NULLABLE |
| eventos_unicos | INT64 | NULLABLE |
| duplicados | INT64 | NULLABLE |

### retail_ops.v_costo_real_mensual (bigquery, 6 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| **mes** | STRING | NULLABLE |
| **servicio** | STRING | NULLABLE |
| **usd_neto** | FLOAT64 | NULLABLE |
| **usd_total_mes** | FLOAT64 | NULLABLE |
| **pedidos** | INT64 | NULLABLE |
| **usd_por_pedido** | FLOAT64 | NULLABLE |

### Cloud SQL · retail.users (cloudsql, 3 columnas)

| Columna | Tipo | Modo | Descripción |
|---|---|---|---|
| **user_id** | TEXT | REQUIRED | PK |
| **nombre** | TEXT | REQUIRED | — |
| **created_at** | TIMESTAMP | NULLABLE | — |

### Cloud SQL · retail.articles (cloudsql, 11 columnas)

| Columna | Tipo | Modo | Descripción |
|---|---|---|---|
| **article_id** | TEXT | REQUIRED | PK |
| **prod_name** | TEXT | REQUIRED | — |
| **product_type** | TEXT | NULLABLE | — |
| **product_group** | TEXT | NULLABLE | — |
| **index_group** | TEXT | NULLABLE | — |
| **section** | TEXT | NULLABLE | — |
| **colour** | TEXT | NULLABLE | — |
| **detail_desc** | TEXT | NULLABLE | — |
| **price_cents** | INTEGER | REQUIRED | Precio demo sintético |
| **popularity** | INTEGER | REQUIRED | Ventas reales 28 días |
| **stock** | INTEGER | REQUIRED | Stock simulado |

### Cloud SQL · retail.cart_items (cloudsql, 3 columnas)

| Columna | Tipo | Modo | Descripción |
|---|---|---|---|
| **user_id** | TEXT | REQUIRED | — |
| **article_id** | TEXT | REQUIRED | — |
| **qty** | INTEGER | REQUIRED | 1 a 10 |

### Cloud SQL · retail.orders (cloudsql, 5 columnas)

| Columna | Tipo | Modo | Descripción |
|---|---|---|---|
| **order_id** | TEXT | REQUIRED | PK |
| **user_id** | TEXT | REQUIRED | — |
| **total_cents** | INTEGER | REQUIRED | — |
| **idempotency_key** | TEXT | REQUIRED | Única: evita doble cobro |
| **created_at** | TIMESTAMP | NULLABLE | — |

## Cloud SQL · retail.order_lines (cloudsql, 4 columnas)

| Columna | Tipo | Modo |
|---|---|---|
| **order_id** | TEXT | REQUIRED |
| **article_id** | TEXT | REQUIRED |
| **qty** | INTEGER | REQUIRED |
| **price_cents** | INTEGER | REQUIRED |

## Cloud SQL · retail.wallet_ledger (cloudsql, 6 columnas)

| Columna | Tipo | Modo | Descripción |
|---|---|---|---|
| **entry_id** | TEXT | REQUIRED | PK |
| **user_id** | TEXT | REQUIRED | — |
| **amount_cents** | INTEGER | REQUIRED | — |
| **kind** | TEXT | REQUIRED | demo_topup \| purchase |
| **ref** | TEXT | NULLABLE | — |
| **created_at** | TIMESTAMP | NULLABLE | — |

# Anexo D. Modelo estrella (Gold)

Dataset `retail_gold`. Hechos en dorado, dimensiones en azul y tabla de recomendación en morado. Las flechas indican la clave de unión.

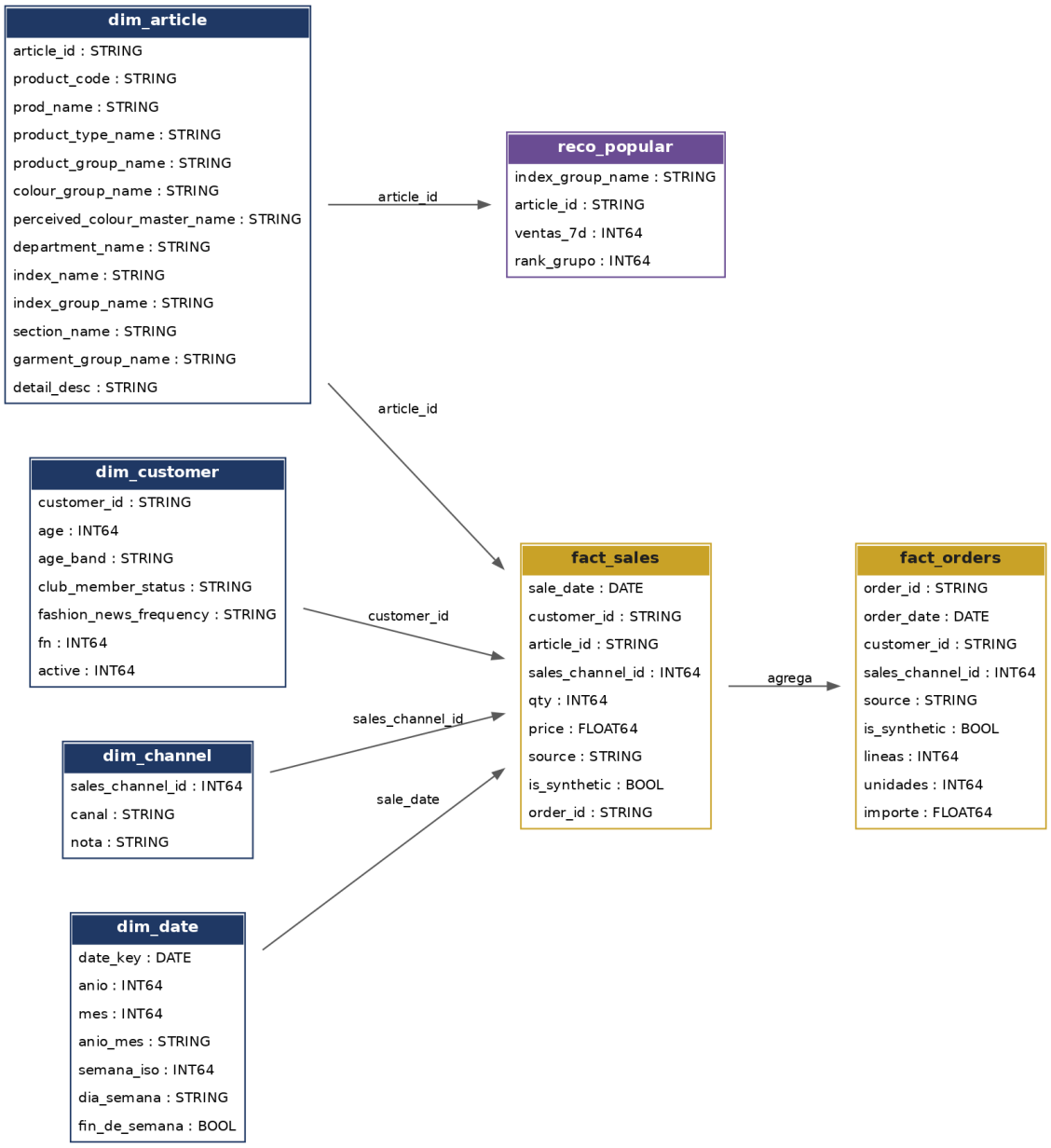



**Figura 14.** *Modelo estrella con todas las columnas y tipos.*

# Anexo E. Matriz de permisos IAM

| Principal | Rol | Alcance | Motivo | Gestión |
|---|---|---|---|---|
| sa:ingest | roles/storage.objectAdmin | bucket:raw | Escribir Bronze | excel |
| sa:ingest | roles/pubsub.publisher | topic:events | Publicar eventos | excel |
| sa:pipeline | roles/dataproc.worker | proyecto | Ejecutar Serverless Spark | excel |
| sa:pipeline | roles/dataflow.worker | proyecto | Ejecutar workers de Dataflow | excel |
| sa:pipeline | roles/bigquery.jobUser | proyecto | Ejecutar consultas y cargas | excel |
| sa:pipeline | roles/storage.objectViewer | bucket:raw | Leer Bronze | excel |
| sa:pipeline | roles/storage.objectAdmin | bucket:lake | Escribir Silver/Gold y temporales | excel |
| sa:pipeline | roles/bigquery.dataEditor | dataset:silver | Escribir Silver | excel |
| sa:pipeline | roles/bigquery.dataEditor | dataset:gold | Escribir Gold | excel |
| sa:pipeline | roles/bigquery.dataEditor | dataset:ops | Métricas de streaming y DLQ | excel |
| sa:pipeline | roles/pubsub.subscriber | subscription:dataflow | Consumir eventos | excel |
| sa:pipeline | roles/pubsub.viewer | subscription:dataflow | Dataflow consulta la suscripción | excel |
| sa:api | roles/bigquery.jobUser | proyecto | Consultar el catálogo Gold | excel |
| sa:api | roles/bigquery.dataViewer | dataset:gold | Leer Gold | excel |
| sa:api | roles/aiplatform.user | proyecto | Invocar Gemini | excel |
| sa:api | roles/cloudsql.client | proyecto | Conectarse a Cloud SQL | excel |
| sa:api | roles/secretmanager.secretAccessor | proyecto | Leer URL de base y token admin | excel |
| sa:api | roles/pubsub.publisher | topic:events | Publicar compras de la web | excel |
| sa:api | roles/storage.objectViewer | bucket:raw | Servir imágenes H&M | excel |
| sa:deployer | roles/run.admin | proyecto | Desplegar Cloud Run | excel |
| sa:deployer | roles/artifactregistry.writer | proyecto | Publicar imágenes | excel |
| sa:deployer | roles/iam.serviceAccountUser | cuenta:api | Desplegar revisiones que corren como sa-retail-api | excel |
| compute-default | roles/cloudbuild.builds.builder | proyecto | Cloud Build (ops api) construye la imagen | excel |
| pubsub-agent | roles/bigquery.dataEditor | dataset:bronze | Suscripción BigQuery escribe events_raw | excel |
| pubsub-agent | roles/bigquery.metadataViewer | dataset:bronze | Suscripción BigQuery valida el esquema | excel |
| pubsub-agent | roles/pubsub.publisher | topic:dlq | Reenviar mensajes fallidos a la DLQ | excel |
| pubsub-agent | roles/pubsub.subscriber | subscription:bq | Requisito de dead-letter | excel |
| pubsub-agent | roles/pubsub.subscriber | subscription:dataflow | Requisito de dead-letter | excel |
| github:<github_repo> | roles/iam.workloadIdentityUser | cuenta:deployer | GitHub Actions asume sa-retail-deployer (cicd.tf) | fijo en código |
| usuario que crea el proyecto | roles/owner | proyecto | Lo asigna GCP al crear el proyecto; no se puede otorgar por API | informativo |

## Usuarios iniciales

| Tipo | ID | Nombre | Perfil | Roles GCP / saldo inicial |
|---|---|---|---|---|
| gcp | `estuardo` | Estuardo Oliver | ML / Analytics | bigquery.user, bigquery.dataEditor, aiplatform.user, run.developer, logging.viewer |
| gcp | `winston` | Winston Quispe | Cloud Computing | editor, resourcemanager.projectIamAdmin, iam.serviceAccountUser |
| gcp | `nicole` | Nicole Arenas | Datos + API + Modelado | bigquery.admin, storage.admin, dataproc.editor, run.developer, cloudsql.client, iam.serviceAccountUser |
| gcp | `gianmarco` | Gianmarco Mejia | Dashboard / BI | bigquery.dataViewer, bigquery.jobUser, monitoring.viewer |
| app | `demo` | Usuario demo | Cliente demo | USD 200.00 |
| app | `demo2` | Usuario demo 2 | Cliente demo sin historial (cold-start) | USD 0.00 |
| app | `estuardo` | Estuardo Oliver | Equipo | USD 200.00 |
| app | `winston` | Winston Quispe | Equipo | USD 200.00 |
| app | `nicole` | Nicole Arenas | Equipo | USD 200.00 |
| app | `gianmarco` | Gianmarco Mejia | Equipo | USD 200.00 |
| base_datos | `retail_app` | Usuario de aplicación Cloud SQL | — | — |
| admin_api | `X-Admin-Token` | Token de recargas demo | — | — |

# Anexo F. Parámetros de configuración

## Proyecto

| Clave | Valor | Tipo | Editable | Descripción |
|---|---|---|---|---|
| **project_id** | tranquil-leaf-512618-b9 | texto | Sí | ID único del proyecto GCP (6-30 caracteres, minúsculas). OBLIGATORIO cambiar |
| **region** | us-central1 | texto | Sí | Región única del prototipo (Tier 1, Gemini disponible) |
| **env** | dev | texto | Sí | Ambiente: dev \| demo \| prod (sufijo de recursos y del estado Terraform) |
| **labels** | proyecto=retail-bigdata;curso=utec-bigdata | mapa | Sí | Etiquetas en todos los recursos (clave=valor;...) |

## Costos

| Clave | Valor | Tipo | Editable | Descripción |
|---|---|---|---|---|
| **billing_account** | 013564-3E664C-2B3FED | texto | Sí | Cuenta de facturación XXXXXX-XXXXXX-XXXXXX. Vacío = no crea presupuesto |
| **monthly_budget_usd** | 50 | numero | Sí | Presupuesto mensual con alertas |
| **budget_thresholds** | 0.5;0.9;1.0 | lista_numeros | Sí | Umbrales de alerta (fracción del presupuesto) |
| **budget_topic** | billing-budget-alerts | texto | No | Tópico Pub/Sub que recibe las alertas de presupuesto |
| **billing_export_table** | — | texto | Sí | Tabla del billing export en el dataset ops (gcp_billing_export_resource_v1_...). Vacío = sin costo real |

## Almacenamiento

| Clave | Valor | Tipo | Editable | Descripción |
|---|---|---|---|---|
| **bucket_raw_suffix** | retail-raw | texto | No | Bucket Bronze = <project_id>-<sufijo> |
| **bucket_lake_suffix** | retail-lake | texto | No | Bucket Silver/Gold/tmp = <project_id>-<sufijo> |
| **bucket_tfstate_suffix** | tfstate | texto | No | Bucket del estado Terraform = <project_id>-<sufijo> |
| **tfstate_prefix** | retail | texto | No | Carpeta del estado dentro del bucket (se agrega /<env>) |
| **images_nearline_days** | 30 | numero | Sí | Días para pasar imágenes a Nearline |
| **noncurrent_version_days** | 30 | numero | Sí | Días de retención de versiones antiguas de objetos |
| **force_destroy_buckets** | false | booleano | Sí | true = destroy borra buckets con datos |
| **local_raw_dir** | data/raw | texto | Sí | Carpeta local con los CSV e imágenes H&M |

## Streaming

| Clave | Valor | Tipo | Editable | Descripción |
|---|---|---|---|---|
| **pubsub_topic** | retail-events | texto | No | Tópico de eventos |
| **pubsub_dlq_topic** | retail-events-dlq | texto | No | Tópico de mensajes fallidos |
| **pubsub_bq_subscription** | retail-events-to-bq | texto | No | Suscripción directa a BigQuery (ruta económica) |
| **pubsub_bq_table** | bronze.events_raw | texto | No | Tabla destino de la suscripción (dataset_clave.tabla, ver hoja Tablas) |
| **pubsub_dataflow_subscription** | retail-events-dataflow | texto | No | Suscripción que consume Dataflow |
| **pubsub_dlq_subscription** | retail-events-dlq-sub | texto | No | Suscripción que retiene los mensajes fallidos |
| **pubsub_max_delivery_attempts** | 5 | numero | Sí | Reintentos antes de enviar a la DLQ |
| **pubsub_retention** | 86400s | texto | Sí | Retención de mensajes no confirmados |
| **dataflow_job_name** | retail-events | texto | No | Nombre del job streaming |
| **dataflow_machine_type** | n1-standard-2 | texto | Sí | Tipo de máquina de los workers |
| **dataflow_max_workers** | 3 | numero | Sí | Máximo de workers (tope de costo) |
| **replay_target_start** | 2026-11-01 | texto | Sí | Fecha base de los eventos 2026 |
| **replay_events_per_sec** | 50 | numero | Sí | Ritmo del replay |
| **replay_limit** | 5000 | numero | Sí | Eventos por ejecución de replay |

## Procesamiento batch

| Clave | Valor | Tipo | Editable | Descripción |
|---|---|---|---|---|
| **dataproc_runtime** | 2.2 | texto | No | Versión de Serverless for Apache Spark |
| **dataproc_subnet** | default | texto | No | Subred con Private Google Access |

## Base transaccional (Cloud SQL)

| Clave | Valor | Tipo | Editable | Descripción |
|---|---|---|---|---|
| **cloudsql_enabled** | true | booleano | Sí | false = sin Cloud SQL ni Cloud Run (ahorra USD 8-11/mes) |
| **cloudsql_instance** | retail-pg | texto | No | Prefijo de la instancia (se agrega -<env>-<sufijo aleatorio>) |
| **cloudsql_version** | POSTGRES_15 | texto | No | Motor |
| **cloudsql_tier** | db-f1-micro | texto | Sí | Tamaño (db-f1-micro, db-g1-small, db-custom-1-3840) |
| **cloudsql_ha** | false | booleano | Sí | true = alta disponibilidad regional (duplica costo) |
| **cloudsql_disk_gb** | 10 | numero | Sí | Disco SSD |
| **cloudsql_db_name** | retail | texto | No | Base de datos de la aplicación |
| **cloudsql_user** | retail_app | texto | No | Usuario de base (contraseña generada en Secret Manager) |
| **cloudsql_backup_start** | 06:00 | texto | Sí | Hora UTC del backup diario (PITR activo) |
| **cloudsql_deletion_protection** | false | booleano | Sí | true en producción |

## API y web (Cloud Run)

| Clave | Valor | Tipo | Editable | Descripción |
|---|---|---|---|---|
| **artifact_repo** | retail | texto | No | Repositorio Docker en Artifact Registry |
| **cloudrun_service** | retail-api | texto | No | Servicio Cloud Run (API + web) |
| **catalog_job** | retail-catalog | texto | No | Cloud Run Job que carga el catálogo Gold -> Cloud SQL |
| **api_image** | us-docker.pkg.dev/cloudrun/container/hello | texto | No | Imagen inicial (luego ops api o GitHub la reemplazan) |
| **api_min_instances** | 0 | numero | Sí | 0 = escala a cero; 1 en la demo evita arranque en frío |
| **api_max_instances** | 3 | numero | Sí | Tope de instancias |
| **api_cpu** | 1 | texto | Sí | vCPU por instancia |
| **api_memory** | 512Mi | texto | Sí | Memoria por instancia |
| **api_public** | true | booleano | Sí | true = web accesible sin login |
| **assistant_mode** | gemini | texto | Sí | gemini \| rules (rules = sin Vertex AI, costo 0) |
| **gemini_model** | gemini-3.1-flash-lite | texto | Sí | Modelo Vertex AI (2.5-flash-lite se retiró en Vertex el 20-oct-2026) |
| **gemini_location** | us-central1 | texto | Sí | Región de Vertex AI |
| **price_demo_factor** | 500 | numero | Sí | Precio demo = precio normalizado H&M x factor (sintético) |
| **stock_demo** | 50 | numero | Sí | Stock simulado por artículo al cargar el catálogo |
| **demo_topup_max_cents** | 50000 | numero | Sí | Máximo por recarga demo |
| **admin_token_local** | (local, no productivo) | texto | Sí | Token de recargas SOLO para ejecución local (en GCP se genera en Secret Manager) |
| **secret_db_url** | retail-db-url | texto | No | Secreto con la URL de conexión a Cloud SQL |
| **secret_admin_token** | retail-admin-token | texto | No | Secreto con el token de recargas demo |

## CI/CD

| Clave | Valor | Tipo | Editable | Descripción |
|---|---|---|---|---|
| **github_repo** | estuardoo/STYLUS | texto | Sí | owner/repo para desplegar desde GitHub Actions sin llaves. Vacío = sin WIF |
| **wif_pool** | github | texto | No | Prefijo del Workload Identity Pool (se agrega -<env>) |
| **deployer_sa_key** | deployer | texto | No | Clave (hoja Cuentas_Servicio) de la cuenta que usa GitHub |

# Anexo G. APIs de Google Cloud habilitadas

| API | Motivo |
|---|---|
| serviceusage.googleapis.com | Gestionar APIs |
| cloudresourcemanager.googleapis.com | Proyecto e IAM |
| iam.googleapis.com | Cuentas de servicio |
| iamcredentials.googleapis.com | Tokens para WIF |
| sts.googleapis.com | Workload Identity Federation |
| compute.googleapis.com | Red para Dataproc/Dataflow |
| storage.googleapis.com | Data Lake |
| bigquery.googleapis.com | Data Warehouse |
| pubsub.googleapis.com | Streaming |
| dataflow.googleapis.com | Procesamiento streaming |
| dataproc.googleapis.com | Spark serverless |
| run.googleapis.com | API, web y jobs |
| cloudbuild.googleapis.com | Construir la imagen de la API |
| artifactregistry.googleapis.com | Imágenes Docker |
| aiplatform.googleapis.com | Vertex AI Gemini |
| sqladmin.googleapis.com | Cloud SQL |
| secretmanager.googleapis.com | Secretos |
| billingbudgets.googleapis.com | Presupuestos |
| cloudbilling.googleapis.com | Facturación |
| monitoring.googleapis.com | Métricas y alertas |
| logging.googleapis.com | Logs |

# Anexo H. Glosario

| Término | Significado |
|---|---|
| **Medallion** | Organización del lago en Bronze (crudo), Silver (limpio) y Gold (modelo de negocio) |
| **Modelo estrella** | Tabla de hechos rodeada de dimensiones; optimizado para BI |
| **DLQ** | Dead Letter Queue: destino de mensajes que no se pudieron procesar |
| **Idempotencia** | Repetir una operación produce el mismo resultado sin duplicar |
| **WIF** | Workload Identity Federation: GitHub se autentica en GCP sin llaves |
| **MAP@12** | Mean Average Precision de las 12 primeras recomendaciones |
| **PITR** | Point-in-time recovery de Cloud SQL |
| **Cold-start** | Cliente sin historial; recibe recomendaciones por popularidad |